

# Algorytmy Zaawansowane

Opracowane na podstawie wykładów Michała Tuczyńskiego

## Spis treści

---

|                                                                                     |           |
|-------------------------------------------------------------------------------------|-----------|
| <b>1 Wykład 2</b>                                                                   | <b>2</b>  |
| 1.1 Notacja asymptotyczna — przypomnienie . . . . .                                 | 2         |
| 1.2 Strategia typu <i>Dziel i Zwyciężaj</i> ( <i>Divide and Conquer</i> ) . . . . . | 3         |
| <b>2 Wykład 3</b>                                                                   | <b>6</b>  |
| 2.1 Algorytm Strasięra . . . . .                                                    | 6         |
| 2.2 Własność optymalnej podstruktury . . . . .                                      | 7         |
| 2.3 Znajdywanie pary najbliższych punktów . . . . .                                 | 7         |
| <b>3 Wykład 4 i 5</b>                                                               | <b>10</b> |
| 3.1 Programowanie dynamiczne . . . . .                                              | 10        |
| 3.2 Algorytm mnożenia macierzy . . . . .                                            | 12        |
| 3.3 Problem mnożenia ciągu macierzy . . . . .                                       | 12        |
| 3.4 Problem najdłuższego wspólnego podciągu . . . . .                               | 19        |
| <b>4 Wykład 6</b>                                                                   | <b>22</b> |
| 4.1 Algorytmy Zachłanne . . . . .                                                   | 22        |
| <b>5 Wykład 7</b>                                                                   | <b>25</b> |
| 5.1 Kody Huffmana . . . . .                                                         | 25        |
| 5.2 Algorytm Huffmana . . . . .                                                     | 27        |
| <b>6 Wykład 8</b>                                                                   | <b>29</b> |
| 6.1 Dowód optymalności algorytmu Huffmana . . . . .                                 | 29        |
| 6.2 Szeregowanie zadań . . . . .                                                    | 31        |
| <b>7 Wykład 9</b>                                                                   | <b>33</b> |
| 7.1 Matroidy . . . . .                                                              | 33        |
| 7.2 Problem szeregowania zadań . . . . .                                            | 35        |
| <b>8 Wykład 10</b>                                                                  | <b>35</b> |
| 8.1 Problem szeregowania zadań (cd.) . . . . .                                      | 35        |
| 8.2 Algorytmy aproksymacyjne . . . . .                                              | 37        |
| 8.3 Problem pokrycia wierzchołkowego . . . . .                                      | 37        |
| <b>9 Wykład 11</b>                                                                  | <b>39</b> |
| 9.1 Algorytm Approx Vertex Cover (cd.) . . . . .                                    | 39        |
| 9.2 Problem pokrycia zbioru . . . . .                                               | 39        |
| 9.3 Schematy aproksymacyjne . . . . .                                               | 42        |
| <b>10 Wykład 12</b>                                                                 | <b>42</b> |
| 10.1 Problem sumy podzbiorów . . . . .                                              | 42        |
| 10.2 NP-zupełność problemu sumy podzbiorów . . . . .                                | 45        |
| <b>11 Wykład 13</b>                                                                 | <b>46</b> |
| 11.1 Problem sumy podzbioru . . . . .                                               | 46        |
| 11.2 Najliczniejsze skojarzenie w grafach . . . . .                                 | 49        |
| <b>12 Wykład 14</b>                                                                 | <b>50</b> |

## Wykład 2

### Notacja asymptotyczna — przypomnienie

$$f, g : \mathbb{N} \rightarrow \mathbb{R}_+$$

$$g(n) = \mathcal{O}(f(n)) \Leftrightarrow \exists_{c \in \mathbb{R}_+} \exists_{n_0 \in \mathbb{N}_+} \forall_{n > n_0} g(n) \leq c \cdot f(n)$$

$$g(n) = \Omega(f(n)) \Leftrightarrow \exists_{c \in \mathbb{R}_+} \exists_{n_0 \in \mathbb{N}_+} \forall_{n > n_0} c \cdot f(n) \leq g(n)$$

$$g(n) = \Theta(f(n)) \Leftrightarrow \exists_{c_1, c_2 \in \mathbb{R}_+} \exists_{n_0 \in \mathbb{N}_+} \forall_{n > n_0} c_1 \cdot f(n) \leq g(n) \leq c_2 \cdot f(n)$$

$$g(n) = \mathcal{O}(f(n)) \Leftrightarrow f(n) = \Omega(g(n))$$

$$g(n) = \Theta(f(n)) \Leftrightarrow (g(n) = \mathcal{O}(f(n)) \wedge g(n) = \Omega(f(n)))$$

$$n^a = \mathcal{O}(n^b) \text{ gdy } 0 < a \leq b$$

$$\log n = \mathcal{O}(n^\epsilon) \text{ gdy } \epsilon > 0$$

$$a^n = \mathcal{O}(b^n) \text{ gdy } 1 < a \leq b$$

$$\log_a n = \Theta(\log_b n) \text{ gdy } a, b > 1$$

$$n^a = \mathcal{O}(b^n) \text{ gdy } 0 < a, b > 1$$

$$T(n) = 5n^3 + 2n^2 + 100n \log n \underset{n \rightarrow \infty}{\approx} 5n^3 = \Theta(n^3)$$

### Procedura rekurencyjna

$$n! = \begin{cases} 1 & \text{gdy } n = 0 \\ n \cdot (n-1)! & \text{gdy } n \geq 1 \end{cases}$$

$$\text{Oczywiste jest, że } n! = \prod_{i=1}^n i$$

**Ciąg Fibonacciego**  $F_n$  - ciąg Fibonacciego.

$$F_n = \begin{cases} 0 & \text{gdy } n = 0 \\ 1 & \text{gdy } n = 1 \\ F_{n-1} + F_{n-2} & \text{gdy } n \geq 2 \end{cases}$$

#### Algorytm 1: Rekurencyjna konstrukcja ciągu Fibonacciego

```
function FIBB(n)
1:  if n = 0 then return 0
2:  if n = 1 then return 1
3:  return FIBB(n-1) + FIBB(n-2)
```

Wywołanie rekurencyjne dla (typowo łatwiejszych i mniejszych) instancji tego samego problemu.

## Strategia typu *Dziel i Zwyciężaj* (*Divide and Conquer*)

Algorytm typu *dziel i zwyciężaj* składa się z trzech części

1. *Dziel* – problem jest dzielony na mniejsze podproblemy
2. *Zwyciężaj* – podproblemy są rozwiązywane rekurencyjnie
3. *Połącz* – rozwiązanie całego problemu jest konstruowane z rozwiązań podproblemów

Zakładamy, że (zazwyczaj) w fazie *dziel* problem jest dzielony na co najmniej dwa podproblemy i podproblemy są rozłączne.

### Przykład 1.1 (Mnożenie liczb $n$ -cyfrowych).

$x, y$  – liczby naturalne  $n$ -cyfrowe.

Aby policzyć  $x \cdot y$  można użyć mnożenia pisemnego (jak na kartce). Tym sposobem musimy wykonać  $\mathcal{O}(n^2)$  mnożeń cyfr. Spróbujemy znaleźć lepszą metodę. Dzielimy  $x$  i  $y$  na „dwie połowy”. Zakładamy, dla uproszczenia, że  $n$  jest parzyste.

$$x = x_1 x_2 \cdots x_n = \underbrace{x_1 x_2 \cdots x_{n/2}}_{x_L} \underbrace{x_{n/2+1} \cdots x_n}_{x_R} = 10^{n/2} x_L + x_R$$

podobnie dla  $y$ :

$$y = \cdots = 10^{n/2} y_L + y_R$$

Wymnażamy:

$$x \cdot y = (10^{n/2} x_L + x_R) \cdot (10^{n/2} y_L + y_R) = 10^n x_L y_L + 10^{n/2} (x_L y_R + x_R y_L) + x_R y_R$$

Złożoność czasowa algorytmu – ozn.  $T(n)$ .

- Podział  $x, y$  na  $x_L, x_R, y_L, y_R$  zajmie  $\mathcal{O}(n)$  czasu.
- Dodanie liczb  $n$ -cyfrowych zajmie  $\mathcal{O}(n)$  czasu.
- Mnożenie przez  $10^n$  jest równoważne dopisaniu  $n$  zer więc też  $\mathcal{O}(n)$ .

$$T(n) = 4 \cdot T(n/2) + \Theta(n)$$

Można lepiej, bo możemy zauważyć, że  $x_L y_R + x_R y_L = (x_L - x_R) \cdot (y_R - y_L) + x_L y_L + x_R y_R$ , więc potrzebujemy policzyć tylko 3 iloczyny:  $x_L y_L$ ,  $x_R y_R$ ,  $(x_L - x_R) \cdot (y_R - y_L)$ . To daje  $T(n) = 3 \cdot T\left(\frac{n}{2}\right) + \Theta(n)$

### Twierdzenie 1.2 (CLRS - Uniwersalne Twierdzenie o Rekurencji).

Niech  $a \geq 1$ ,  $b > 1$  będą stałymi,  $f(n)$  funkcją i  $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$  (gdzie  $\frac{n}{b}$  znaczy  $\left\lfloor \frac{n}{b} \right\rfloor$  lub  $\left\lceil \frac{n}{b} \right\rceil$ ). Wtedy

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{gdy } f(n) = \mathcal{O}(n^{\log_b a - \epsilon}) \text{ dla pewnego } \epsilon > 0 \\ \Theta(n^{\log_b a} \log n) & \text{gdy } f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)) & \text{gdy } \begin{matrix} f(n) = \Omega(n^{\log_b a + \epsilon}) \text{ dla pewnego } \epsilon > 0 \\ \text{a } f(n/b) \leq c \cdot f(n) \text{ dla pewnej stałej } c < 1 \text{ dla dużych } n \end{matrix} \end{cases}$$

**Lemat 1.3.** Niech  $a \geq 1$ ,  $b > 1$  będą stałymi i niech  $f(n)$  będzie nieujemną funkcją zdefiniowaną dla potęg  $b$ . Jeśli

$$T(n) = \begin{cases} \Theta(1) & \text{gdy } n = 1 \\ a \cdot T\left(\frac{n}{b}\right) + f(n) & \text{gdy } n = b^i, i \geq 1 \end{cases}$$

to

$$T(n) = \Theta\left(n^{\log_b a}\right) + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

*Dowód.*

$$a^{\log_b n} = \left(b^{\log_b a}\right)^{\log_b n} = b^{\log_b a \cdot \log_b n} = b^{\log_b n \cdot \log_b a} = \left(b^{\log_b n}\right)^{\log_b a} = n^{\log_b a} \quad (1)$$

$$\begin{aligned} T(n) &= f(n) + a \cdot T\left(\frac{n}{b}\right) = f(n) + a \left( f\left(\frac{n}{b}\right) + aT\left(\frac{n}{b^2}\right) \right) \\ &= f(n) + af\left(\frac{n}{b}\right) + a^2 T\left(\frac{n}{b^2}\right) = f(n) + af\left(\frac{n}{b}\right) + a^2 \left( f\left(\frac{n}{b^2}\right) + aT\left(\frac{n}{b^3}\right) \right) \\ &= f(n) + af\left(\frac{n}{b}\right) + a^2 f\left(\frac{n}{b^2}\right) + \dots + a^{\log_b n - 1} f\left(\frac{n}{b^{\log_b n}}\right) + a^{\log_b n} T(1) \\ &\stackrel{1}{=} \dots + a^{\log_b n} T(1) = \dots + n^{\log_b a} T(1) = \Theta\left(n^{\log_b a}\right) + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) \end{aligned}$$

□

Przeprowadzimy teraz dowód uniwersalnego twierdzenia o rekurencji (1.2). Ograniczymy się do przypadku, gdy  $n$  jest potęgą  $b$ . Pozostały przypadek jest bardziej techniczny.

*Dowód.* Rozważmy wszystkie przypadki:

1. Gdy  $f(n) = \mathcal{O}\left(n^{\log_b a - \epsilon}\right)$ . Wtedy

$$\begin{aligned} f\left(\frac{n}{b^j}\right) &= \mathcal{O}\left(\left(\frac{n}{b^j}\right)^{\log_b a - \epsilon}\right) \\ \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) &= \sum_{j=0}^{\log_b n - 1} a^j \mathcal{O}\left(\left(\frac{n}{b^j}\right)^{\log_b a - \epsilon}\right) = \mathcal{O}\left(\underbrace{\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \epsilon}}_{\blacktriangle}\right) \\ \blacktriangle &= \sum_{j=0}^{\log_b n - 1} a^j \frac{n^{\log_b a - \epsilon}}{b^{j(\log_b a - \epsilon)}} = n^{\log_b a - \epsilon} \sum_{j=0}^{\log_b n - 1} a^j \frac{b^{j\epsilon}}{b^{j \log_b a}} \\ &= n^{\log_b a - \epsilon} \sum_{j=0}^{\log_b n - 1} \left(\frac{a \cdot b^\epsilon}{\underbrace{b^{\log_b a}}_{=a}}\right)^j = n^{\log_b a - \epsilon} \sum_{j=0}^{\log_b n - 1} b^{\epsilon j} \quad (\text{wzór na ciąg geometryczny}) \end{aligned}$$

$$\begin{aligned}
 &= n^{\log_b a - \epsilon} \frac{b^{\epsilon \log_b n} - 1}{b^\epsilon - 1} = n^{\log_b a - \epsilon} \underbrace{\frac{n^\epsilon - 1}{b^\epsilon - 1}}_{\text{stała}} \\
 &= n^{\log_b a - \epsilon} \mathcal{O}(n^\epsilon - 1) = \mathcal{O}(n^{\log_b a - \epsilon} \cdot n^\epsilon - 1) = \mathcal{O}(n^{\log_b a})
 \end{aligned}$$

Z lematu:  $T(n) = \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) + \Theta(n^{\log_b a}) = \mathcal{O}(n^{\log_b a}) + \Theta(n^{\log_b a}) = \Theta(n^{\log_b a})$

2. Gdy  $f(n) = \Theta(n^{\log_b a})$ . Wtedy:

$$\begin{aligned}
 f\left(\frac{n}{b^j}\right) &= \Theta\left(\left(\frac{n}{b^j}\right)^{\log_b a}\right) \\
 \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) &= \sum_{j=0}^{\log_b n - 1} a^j \Theta\left(\left(\frac{n}{b^j}\right)^{\log_b a}\right) = \Theta\left(\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a}\right) \\
 &= \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n - 1} a^j \frac{1}{b^{j \log_b a}}\right) = \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n - 1} a^j \underbrace{\frac{1}{b^{\log_b a^j}}}_{=a^j}\right) \\
 &= \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n - 1} 1\right) = \Theta(n^{\log_b a} \log_b n) \\
 \text{Z lematu: } T(n) &= \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) + \Theta(n^{\log_b a}) = \mathcal{O}(n^{\log_b a} \log n) + \Theta(n^{\log_b a}) = \Theta(n^{\log_b a} \log n)
 \end{aligned}$$

3. Gdy  $f(n) = \Omega(n^{\log_b a + \epsilon})$  oraz  $a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n)$  dla jakiegoś  $c < 1$  i odpowiednio dużych  $n$ .

Z założenia  $f\left(\frac{n}{b}\right) \leq \frac{c}{a} f(n)$ , więc drogą iteracji:

$$\begin{aligned}
 f\left(\frac{n}{b^j}\right) &\stackrel{(\star)}{\leq} \left(\frac{c}{a}\right)^j f(n) = \frac{c^j}{a^j} f(n) \\
 (\star) \quad f\left(\frac{n}{b^j}\right) &\leq \frac{c}{a} f\left(\frac{n}{b^{j-1}}\right) \leq \left(\frac{c}{a}\right)^2 f\left(\frac{n}{b^{j-2}}\right) \leq \dots \leq \left(\frac{c}{a}\right)^j f(n) \\
 \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) &\leq \sum_{j=0}^{\log_b n - 1} \cancel{a^j} \frac{c^j}{\cancel{a^j}} f(n) + \mathcal{O}(1) \leq f(n) \sum_{j=0}^{\log_b n - 1} c^j + \mathcal{O}(1) \\
 &\leq f(n) \sum_{j=0}^{\infty} c^j + \mathcal{O}(1) \leq f(n) \frac{1}{1-c} + \mathcal{O}(1) \leq f(n) \mathcal{O}(1) = \mathcal{O}(f(n)) \\
 \text{Z drugiej strony: } \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) &= a^0 f\left(\frac{n}{b^0}\right) + \underbrace{\sum_{j=1}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)}_{\geq 0} = \Omega(f(n))
 \end{aligned}$$

Z lematu:  $T(n) = \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) + \Theta(n^{\log_b a}) = \Theta(f(n)) + \Theta(n^{\log_b a}) = \Theta(f(n))$

□

**Wniosek 1.4.** Niech  $a \geq 1$ ,  $b > 1$  będą stałymi, a  $f(n)$  funkcją taką, że  $f(n) = \Theta(n^k)$ . Niech  $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$  (gdzie  $\frac{n}{b}$  znaczy  $\left\lfloor \frac{n}{b} \right\rfloor$  lub  $\left\lceil \frac{n}{b} \right\rceil$ ). Wtedy

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{gdzy } f(n) = \mathcal{O}(n^{\log_b a - \epsilon}) \implies k < \log_b a \iff b^k < a \\ \Theta(n^k \log n) & \text{gdzy } f(n) = \Theta(n^{\log_b a}) \implies k = \log_b a \iff b^k = a \\ \Theta(n^k) & \text{gdzy } f(n) = \Omega(n^{\log_b a + \epsilon}) \implies k > \log_b a \iff b^k > a \end{cases}$$

Wróćmy teraz do naszego problemu mnożenia liczb  $n$ -cyfrowych. Drugi algorytm ma złożoność:

$$T(n) = 4 \cdot T\left(\frac{n}{2}\right) + \Theta(n^1) \quad a = 4, b = 2, k = 1$$

$$4 > 2^1 \Rightarrow T(n) = \Theta(n^{\log_2 4}) = \Theta(n^2)$$

Trzeci algorytm ma złożoność:

$$T(n) = 3 \cdot T\left(\frac{n}{2}\right) + \Theta(n^1) \quad a = 3, b = 2, k = 1$$

$$3 > 2^1 \Rightarrow T(n) = \Theta(n^{\log_2 3}) = \Theta(n^{1.59})$$

## Wykład 3

### Przykład 2.1 (Mnożenie macierzy).

$A, B$  - macierze  $n \times n$

$$A \cdot B = C = ?$$

Najpopularniejszy sposób mnożenia dwóch macierzy  $n \times n$  działa w czasie  $\mathcal{O}(n^3)$ . Musimy obliczyć  $n^2$  wartości i obliczenie każdej zajmuje liniowy czas.

$$B = \begin{bmatrix} \vdots \end{bmatrix} \quad A = \begin{bmatrix} \cdots \end{bmatrix} \quad \begin{bmatrix} \times \end{bmatrix} = C$$

### Algorytm Strasiera

Zakładamy, że  $n$  jest potęgą 2 (dla przejrzystości). Dzielimy  $A$  i  $B$  na 4 macierze  $\frac{n}{2} \times \frac{n}{2}$ :

$$A = \left[ \begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array} \right], \quad B = \left[ \begin{array}{c|c} B_{11} & B_{12} \\ \hline B_{21} & B_{22} \end{array} \right]$$

$$A \cdot B = \left[ \begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array} \right] \cdot \left[ \begin{array}{c|c} B_{11} & B_{12} \\ \hline B_{21} & B_{22} \end{array} \right] = \left[ \begin{array}{c|c} C_{11} & C_{12} \\ \hline C_{21} & C_{22} \end{array} \right]$$

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}$$

$$C_{12} = A_{11}B_{12} + A_{12}B_{22}$$

$$C_{21} = A_{21}B_{11} + A_{22}B_{21}$$

$$C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

Zamieniliśmy 1 mnożenie macierzy  $n \times n$  na 8 mnożeń macierzy  $\frac{n}{2} \times \frac{n}{2}$  + podział i dodawanie ( $\mathcal{O}(n^2)$ ). Stąd mamy zależność rekurencyjną:

$$T(n) = 8 \cdot T\left(\frac{n}{2}\right) + \Theta(n^2) \quad a = 8, b = 2, k = 2$$

$$8 > 2^2 \Rightarrow T(n) = \Theta(n^{\log_2 8}) = \Theta(n^3)$$

Strassen zauważył, że jeśli obliczymy

$$M_1 = (A_{12} - A_{22}) \cdot (B_{21} + B_{22})$$

$$M_2 = (A_{11} + A_{22}) \cdot (B_{11} + B_{22})$$

$$M_3 = (A_{11} - A_{21}) \cdot (B_{11} + B_{12})$$

$$M_4 = (A_{11} + A_{12}) \cdot B_{22}$$

$$M_5 = A_{11} \cdot (B_{12} - B_{22})$$

$$M_6 = A_{22} \cdot (B_{21} - B_{11})$$

$$M_7 = (A_{21} + A_{22}) \cdot B_{11}$$

$$\text{to } C_{11} = M_1 + M_2 - M_4 + M_6$$

$$C_{12} = M_4 + M_5$$

$$C_{21} = M_6 + M_7$$

$$C_{22} = M_2 - M_3 + M_5 - M_7$$

Zatem

$$T(n) = 7 \cdot T\left(\frac{n}{2}\right) + \Theta(n^2) \quad a = 7, b = 2, k = 2$$

$$7 > 2^2 \Rightarrow T(n) = \Theta(n^{\log_2 7}) = \Theta(n^{2.81})$$

### Własność optymalnej podstruktury

Optymalne rozwiązanie problemu jest funkcją optymalnych rozwiązań podproblemów (optymalne rozwiązanie można skonstruować z optymalnych rozwiązań podproblemów). Żeby metoda *dziel i zwyciężaj* działała dobrze muszą być spełnione dwa warunki (przez problem i algorytm):

1. własność optymalnej podstruktury
2. podproblemy są rozłączne

### Znajdywanie pary najbliższych punktów

$Q$  - zbiór  $n \geq 2$  punktów na płaszczyźnie

$$p_1 = (x_1, y_1) \quad p_2 = (x_2, y_2) \quad d(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Problem: znaleźć parę punktów w najmniejszej odległości. Algorytm siłowy (*brute force*), który sprawdza wszystkie pary punktów działa w czasie  $\Theta(n^2)$ , bo mamy  $\binom{n}{2} = \frac{n(n-1)}{2} = \Theta(n^2)$  par punktów. Pokażemy algorytm typu *dziel i zwyciężaj*, który działa w czasie  $\mathcal{O}(n \log n)$ .

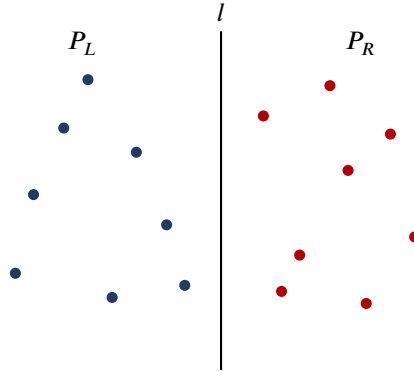
W każdym wywołaniu rekurencyjnym wejście składa się z 3 rzeczy: zbioru  $P \subset Q$  i dwóch tablic  $X$  i  $Y$  które zawierają wszystkie punkty z  $P$ . W tablicy  $X$  punkty są posortowane w porządku niemalejącym względem pierwszej współrzędnej, a w tablicy  $Y$  po drugiej współrzędnej.

Opis algorytmu:

jeśli  $|P| \leq 3$  sprawdzamy wszystkie pary

jeśli  $|P| > 3$

Dziel: Dzielimy punkty  $P$  pionową prostą  $l$  na dwa podzbiory  $P_L$  i  $P_R$  na lewo i na prawo od  $l$  tak, że  $|P_L| = \left\lceil \frac{P}{2} \right\rceil$ ,  $|P_R| = \left\lfloor \frac{P}{2} \right\rfloor$ . Dzielimy tablicę  $X$  na  $X_L$  i  $X_R$  zawierające punkty odpowiednio  $P_L$  i  $P_R$  posortowane w porządku niemalejącym względem pierwszej współrzędnej. Podobnie dzielimy  $Y$  na  $Y_L$  i  $Y_R$  zawierające odpowiednio punkty z  $P_L$  i  $P_R$  posortowane w porządku niemalejącym po drugiej współrzędnej.



Rysunek 1: Podział zbioru  $P$  pionową prostą  $l$  na podzbiory  $P_L$  (na lewo) i  $P_R$  (na prawo) o rozmiarach  $|P_L| = \left\lceil \frac{P}{2} \right\rceil$  oraz  $|P_R| = \left\lfloor \frac{P}{2} \right\rfloor$ .

Zwycięzaj: Wywołujemy rekurencyjnie algorytm dla wejścia  $P_L$ ,  $X_L$ ,  $Y_L$  i  $P_R$ ,  $X_R$ ,  $Y_R$ . Algorytm znajduje pary najbliższych punktów w  $P_L$  i  $P_R$ . Niech  $\delta_L$  i  $\delta_R$  będą najmniejszymi odległościami znalezionymi przez algorytm dla odpowiednio  $P_L$  i  $P_R$ . Niech  $\delta$  będzie mniejszą z nich:  $\delta = \min(\delta_L, \delta_R)$ .

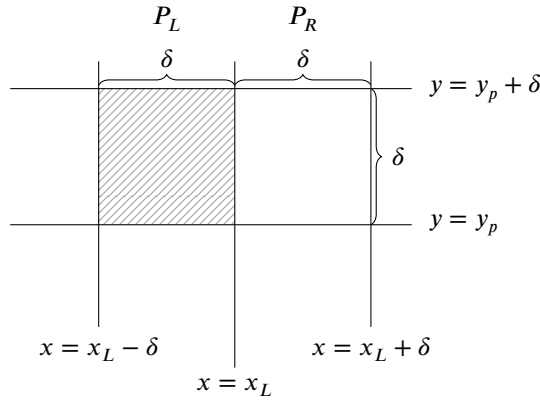
Połącz: Para najbliższych punktów z  $P$  składa się z dwóch punktów z  $P_L$  lub z dwóch punktów z  $P_R$  lub jednego punkty z  $P_L$  i jednego z  $P_R$ .

Musimy sprawdzić czy istnieje para 3-go typu w odległości mniejszej niż  $\delta$ . Jeśli taka para istnieje, wtedy jej punkty są w pionowym pasie o szerokości  $2\delta$  wokół  $l$ .

W celu sprawdzenia czy taka para istnieje wykonujemy następujące czynności:

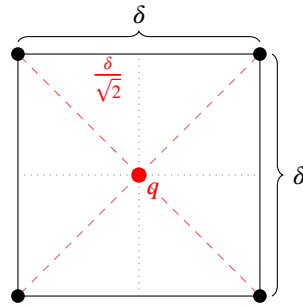
1. Tworzymy tablicę  $Y'$  otrzymaną z  $Y$  poprzez usunięcie punktów spoza pasa.  $Y'$  jest posortowany w porządku niemalejącym względem drugiej współrzędnej.
2. Dla każdego punktu  $p$  z  $Y'$  sprawdzamy odległość między  $p$ , a  $p'$  dla 7 kolejnych punktów  $p'$  z  $Y'$ . Algorytm oblicza najmniejszą odległość  $\delta'$  i zapamiętuje ją i odpowiednią parę punktów.
3. Jeśli  $\delta' < \delta$  to pionowy pas zawiera parę punktów w mniejszej odległości niż pary znalezione w wywołaniach rekurencyjnych. Algorytm zwraca  $\delta'$  i odpowiednią parę punktów. Wpp. zwraca  $\delta$  i odpowiednią parę punktów.

*Dowód poprawności.* Jediną rzeczą, która nie jest oczywista jest fakt, że sprawdzamy tylko 7 następnych punktów po  $p$  w  $Y'$  w fazy łącz. Przypuśćmy, że  $p, q$  są parą punktów w najmniejszej odległości w  $P$  i  $\delta(p, q) = \delta'$ . Niech  $p = (x_p, y_p)$ ,  $q = (x_q, y_q)$ . Bez straty ogólności załóżmy  $y_p \leq y_q$ . Pokażemy, że algorytm znajdzie tę parę punktów. Przypuśćmy, że nie. Wtedy istnieje co najmniej 7 punktów pomiędzy  $p$  i  $q$  w  $Y'$ . Niech  $r = (x_r, y_r)$  będzie jednym z nich. Skoro  $d(p, q) < \delta$  to  $y_q - y_p = |y_q - y_p| \leq \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2} = \delta(p, q) \leq \delta$ . Ponadto  $y_r$  musi być między  $y_p$  i  $y_q$ :  $y_p \leq y_r \leq y_q$ , więc  $y_r - y_p \leq y_q - y_p < \delta$  zatem co najmniej 9 punktów z  $P$  w prostokącie  $2\delta \times \delta$  ograniczonym poziomymi liniami  $y = y_p$ ,  $y = y_p + \delta$ .



Rysunek 2: Prostokąt  $2\delta \times \delta$  ograniczony prostymi  $y = y_p$  oraz  $y = y_p + \delta$ . Zacięniowany lewy kwadrat  $\delta \times \delta$  zawiera co najmniej 5 punktów z  $P$ .

Wtedy w lewym lub prawym kwadracie  $\delta \times \delta$  jest przynajmniej 5 punktów z  $P$ . Przypuśćmy, że to lewy kwadrat. Zatem istnieje kwadrat  $\delta \times \delta$  z 5 punktami z  $P$  w  $P_L$ . Ale wtedy istnieją dwa punkty w  $P_L$  w odległości mniejszej niż  $\delta$ , bo nie da się umieścić w kwadracie  $\delta \times \delta$  5 punktów dla których odległość między dowolnymi dwoma jest mniejsza niż  $\delta$ . Sprzeczność (bo najmniejsza odległość w  $P_L$  to  $\delta_L \geq \delta$ ).



Rysunek 3: W kwadracie  $\delta \times \delta$  nie zmieści się 5 punktów parami oddalonych o co najmniej  $\delta$ . Najlepsze ułożenie to 4 narożniki; dowolny piąty punkt  $q$  jest w odległości co najwyżej  $\frac{\delta}{\sqrt{2}} < \delta$  od któregoś z nich (dwa z 5 punktów trafiają do tej samej ćwiartki  $\frac{\delta}{2} \times \frac{\delta}{2}$ ).

Zatem wystarczy sprawdzić tylko 7 następnych punktów po  $p$  w  $Y'$ . □

**Analiza złożoności czasowej** Na początku musimy posortować tablicę  $X$  i  $Y$ . To zajmuje  $\mathcal{O}(n \log n)$ . Jeśli  $X$  i  $Y$  są już posortowane to podział  $X$  na posortowane  $X_L$  i  $X_R$  oraz  $Y$  analogicznie zajmie  $\mathcal{O}(n)$ .

#### Algorytm 2: Podział posortowanej tablicy $Y$ na $Y_L$ i $Y_R$

```

1:  $length(Y_L) \leftarrow length(Y_R) \leftarrow 0$ 
2: for  $i \leftarrow 1$  to  $length(Y)$  do
3:   if  $Y(i) \in P_L$  then
4:      $length(Y_L) \leftarrow length(Y_L) + 1$ 
5:      $Y_L(length(Y_L)) \leftarrow Y(i)$ 
6:   else
7:      $length(Y_R) \leftarrow length(Y_R) + 1$ 
8:      $Y_R(length(Y_R)) \leftarrow Y(i)$ 

```

Mamy dwa wywołania rekurencyjne dla  $n/2$  punktów i fazę łącz. Tablicę  $Y'$  można otrzymać z  $Y$  w czasie  $\mathcal{O}(n)$  tak, że  $Y'$  jest posortowana. Niech  $x = x_L$  będzie równanie prostej  $l$ .

**Algorytm 3:** Wyznaczanie tablicy  $Y'$  punktów w pasie  $2\delta$  wokół prostej  $l$

```

1:  $length(Y') \leftarrow 0$ 
2: for  $i \leftarrow 1$  to  $length(Y)$  do
3:   if  $x_L - \delta \leq Y(i).x \leq x_L + \delta$  then
4:      $length(Y') \leftarrow length(Y') + 1$ 
5:      $Y'(length(Y')) \leftarrow Y(i)$ 

```

Skoro dla każdego z  $\mathcal{O}(n)$  punktów z  $Y'$  sprawdzamy odległość tylko co najwyżej 7 punktów, to najmniejsza odległość  $\delta'$  punktów z  $Y'$  znajdziemy w czasie  $\mathcal{O}(n)$ .

Oznaczmy złożoność algorytmu bez wstępnego sortowania przez  $T(n)$ . Mamy:

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \Theta(n^1) \qquad a = 2, \quad b = 2, \quad k = 1$$

$$2 = 2^1 \Rightarrow T(n) = \Theta(n \log n)$$

## Wykład 4 i 5

### Programowanie dynamiczne

Czasami gdy zaimplementujemy jakąś formę rekurencyjną dostajemy algorytm działający w czasie wykładniczym. Powodem może być to, że jakieś wartości obliczane są wielokrotnie.

#### Problem 3.1 (liczb Fibonacciego).

$$\begin{cases} F_0 = F_1 = 1 & n = 0, 1 \\ F_n = F_{n-1} + F_{n-2} & n \geq 2 \end{cases} \qquad \text{Zadanie: Obliczyć } n\text{-tą liczbę Fibonacciego}$$

**Algorytm 4:** Algorytm naiwny Fib1

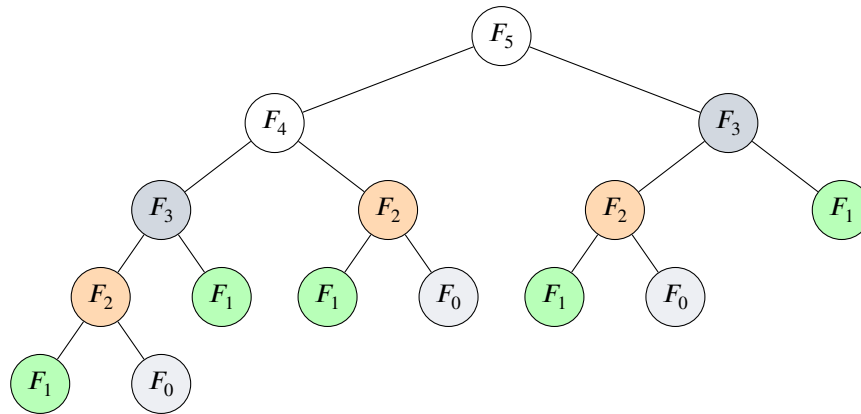
```

function FIB1( $n$ )
1:   if  $n \leq 1$  then return 1
2:   return FIB1( $n - 1$ ) + FIB1( $n - 2$ )

```

Zobaczmy jak jest obliczane np. FIB1(5). W drzewie wywołań poszczególne wartości powtarzają się:

|                       |      |
|-----------------------|------|
| $F(i)$ jest obliczane | razy |
| (1)                   | 5    |
| (2)                   | 3    |
| (3)                   | 2    |
| (4)                   | 1    |
| (5)                   | 1    |



Rysunek 4: Drzewo wywołań rekurencyjnych  $\text{FIB1}(5)$ . Wierzchołki o tym samym kolorze odpowiadają temu samemu podproblemowi, liczonemu wielokrotnie od zera:  $F_3$  jest wyznaczane 2 razy,  $F_2$  – 3 razy, a  $F_1$  – 5 razy.

Niech  $s(n)$  będzie liczbą dodawań wykonanych przez alg. w wywołaniu  $\text{FIB1}(n)$ .

$$\begin{cases} s(0) = s(1) = 0 \\ s(n) = s(n-1) + s(n-2) + 1 & n \geq 2 \end{cases}$$

Niech  $f(n) = s(n) + 1$

Wtedy  $f(0) = f(1) = 1$

$$f(n) = s(n) + 1 =$$

$$= s(n-1) + s(n-2) + 1 + 1 =$$

$$= f(n-1) - 1 + f(n-2) - 1 + 1 + 1 =$$

$$= f(n-1) + f(n-2)$$

Zatem  $f(n) = F_n$ , czyli  $f(n)$  jest  $n$ -tą liczbą Fibonacciego.

Liczby Fibonacciego rosną jak  $n$ -ta potęga złotej liczby, więc

$$f(n) = \Theta(F_n) = \Theta\left(\left(\frac{1+\sqrt{5}}{2}\right)^n\right) = \Omega(1.61^n),$$

gdzie ostatnie oszacowanie jest dolne, bo  $1.61 < \frac{1+\sqrt{5}}{2} \approx 1.618$ .

Stąd  $s(n) = f(n) - 1 = \Omega(1.61^n)$ , więc liczba dodawań rośnie wykładniczo.

$\text{FIB1}$  jest algorytmem wykładniczym.

Oto alg. progr. dynamicznego:

#### Algorytm 5: Algorytm dynamiczny Fib2

```
function FIB2(n)
1:  if  $n \leq 1$  then return 1
2:   $F(0) \leftarrow F(1) \leftarrow 1$ 
```

▷  $F$  - tablica

```

3:   for  $i \leftarrow 2$  to  $n$  do
4:        $F(i) \leftarrow F(i-1) + F(i-2)$ 
5:   return  $F(n)$ 

```

Liczba dodawań wykonanych przez FIB2 w wywołaniu FIB2( $n$ ) wynosi  $n - 1$ . Złożoność czasowa wynosi  $\Theta(n)$ .

### Algorytm mnożenia macierzy

Przypomnijmy, jak mnoży się macierze. Iloczyn  $C = A \cdot B$  macierzy  $A$  rozmiaru  $p \times q$  oraz  $B$  rozmiaru  $q \times r$  jest macierzą  $C$  rozmiaru  $p \times r$ , w której element  $C(i, j)$  to iloczyn skalarny  $i$ -tego wiersza macierzy  $A$  oraz  $j$ -tej kolumny macierzy  $B$ :

$$C(i, j) = \sum_{k=1}^q A(i, k) \cdot B(k, j).$$

Biorąc dla przykładu  $p = 2$ ,  $q = 3$ ,  $r = 2$  i obliczając wyróżniony element  $C(1, 1)$ :

$$\underbrace{\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}}_{A: p \times q} \cdot \underbrace{\begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix}}_{B: q \times r} = \underbrace{\begin{bmatrix} 58 & 64 \\ 139 & 154 \end{bmatrix}}_{C: p \times r}$$

$$C(1, 1) = 1 \cdot 7 + 2 \cdot 9 + 3 \cdot 11 = 7 + 18 + 33 = 58.$$

Kolorowy wiersz macierzy  $A$  i kolorowa kolumna macierzy  $B$  dają kolorowy element macierzy  $C$ ; sumowanie po  $k = 1, \dots, q$  to wewnętrzna pętla algorytmu. Każdy z  $p \cdot r$  elementów macierzy  $C$  wymaga  $q$  mnożeń.

#### Algorytm 6: Naiwne mnożenie macierzy

```

function MATRIXMULTIPLY( $A, B$ )  ▷  $A$  – macierz  $p \times q$ ;  $B$  – macierz  $q \times r$ ;  $C$  – macierz  $p \times r$ 
1:   for  $i \leftarrow 1$  to  $p$  do
2:       for  $j \leftarrow 1$  to  $r$  do
3:            $C(i, j) \leftarrow 0$ 
4:           for  $k \leftarrow 1$  to  $q$  do
5:                $C(i, j) \leftarrow C(i, j) + A(i, k) \cdot B(k, j)$   ▷ operacja dominująca
6:   return  $C$ 

```

Operacją dominującą jest mnożenie w linii 5. Mamy  $p \cdot q \cdot r$  mnożeń. Złożoność jest  $\mathcal{O}(p \cdot q \cdot r)$ .

### Problem mnożenia ciągu macierzy

#### Przykład 3.2 (Mnożenie ciągu macierzy).

$(A_1, A_2, A_3)$  gdzie  $A_1$  – macierz  $5 \times 10$ ,  $A_2$  – macierz  $10 \times 3$ ,  $A_3$  – macierz  $3 \times 2$ . Ile potrzeba mnożeń skalarnych (mnożenia liczb) do obliczenia iloczynu  $A_1 \cdot A_2 \cdot A_3$ ?

$$A_1 \cdot A_2 \cdot A_3 = (A_1 \cdot A_2) \cdot A_3 = A_1 \cdot (A_2 \cdot A_3)$$

I sposób:  $(A_1 \cdot A_2) \cdot A_3 = 5 \cdot 10 \cdot 3 + 5 \cdot 3 \cdot 2 = 150 + 30 = 180$

II sposób:  $A_1 \cdot (A_2 \cdot A_3) = 10 \cdot 3 \cdot 2 + 5 \cdot 10 \cdot 2 = 60 + 100 = 160$

II sposób jest szybszy.

**Problem 3.3 (mnożenia ciągu macierzy).**

Dla danego ciągu macierzy  $(A_1, A_2, \dots, A_n)$ , gdzie  $A_i$  jest macierzą  $p_{i-1} \times p_i$  (cały ciąg wymiarów zapisujemy jednym ciągiem  $p = (p_0, p_1, \dots, p_n)$  – kolejne macierze dzielą wspólny wymiar, więc  $n$  macierzy opisuje  $n + 1$  liczb), znajdź kolejność wykonywania mnożeń minimalizującą liczbę mnożeń skalarnych użytych do obliczenia iloczynu  $A_1 \cdot A_2 \cdot \dots \cdot A_n$ .

Ile jest różnych sposobów obliczenia tego iloczynu? Innymi słowy, na ile sposobów możemy poprawnie wstawić nawiasy w iloczynie  $A_1 \cdot A_2 \cdot \dots \cdot A_n$ ?

**Przykład 3.4 (Kolejności nawiasowania dla  $n = 4$ ).**

$$A_1 \cdot A_2 \cdot A_3 \cdot A_4 \quad (n = 4)$$

- $((A_1 \cdot A_2) \cdot A_3) \cdot A_4$
- $(A_1 \cdot (A_2 \cdot A_3)) \cdot A_4$
- $A_1 \cdot ((A_2 \cdot A_3) \cdot A_4)$
- $A_1 \cdot (A_2 \cdot (A_3 \cdot A_4))$
- $(A_1 \cdot A_2) \cdot (A_3 \cdot A_4)$

Zatem mamy 5 sposobów.

Sprawdźmy czy umiemy rozwiązać ten problem szybko poprzez sprawdzenie wszystkich możliwości. Niech  $P(n)$  – liczba sposobów wstawienia nawiasów w iloczynie  $n$  macierzy. Załóżmy, że ostatnie mnożenie w ciągu  $A_1 \dots A_n$  to mnożenie między  $A_k$  i  $A_{k+1}$ :  $(A_1 \dots A_k)(A_{k+1} \dots A_n)$ . W podciągach lewym i prawym nawiasy są rozmieszczane niezależnie od siebie.

Stąd rekurencja:

$$\begin{cases} P(n) = \sum_{k=1}^{n-1} P(k) \cdot P(n-k) \\ P(1) = 1 \end{cases}$$

Jest to znana formuła rekurencyjna a rozwiązanie to przesunięta liczba Catalana:

$$P(n) = C(n-1), \text{ gdzie } C(n) = \frac{1}{n+1} \binom{2n}{n} \text{ jest } n\text{-tą liczbą Catalana.}$$

Wiadomo, że  $C(n) = \Omega\left(\frac{4^n}{n^{3/2}}\right)$ . Zatem  $P(n)$  jest wykładniczy względem  $n$ . Chcemy znaleźć szybki algorytm.

**Rozwiązanie z optymalną podstrukturą:** Zauważmy że jeśli optymalne rozwiązanie jest postaci  $(A_1 \dots A_k)(A_{k+1} \dots A_n)$  dla jakiegoś  $k \in \{1, \dots, n-1\}$ , to w podciągach  $A_1 \dots A_k$  i  $A_{k+1} \dots A_n$  rozwiązanie nawiasów musi być optymalne. Wpp. lepsze rozmieszczenie nawiasów w  $A_1 \dots A_k$  lub  $A_{k+1} \dots A_n$  dałoby lepsze rozmieszczenie nawiasów w całym ciągu.

Rozwiązanie optymalne zawiera rozwiązania optymalne podproblemów (własność optymalnej podstruktury). Podproblemami w naszym problemie są problemy optymalnego rozmieszczenia nawiasów w ciągach  $A_i A_{i+1} \dots A_j$  dla  $1 \leq i \leq j \leq n$ .

Niech  $m(i, j)$  oznacza minimalną liczbę mnożeń skalarnych potrzebnych do obliczenia tego iloczynu  $A_i \dots A_j$  dla  $1 \leq i \leq j \leq n$ . Chcemy obliczyć  $m(1, n)$ .

$$m(i, i) = 0$$

Optymalny sposób rozmieszczenia nawiasów jest postaci  $(A_i \dots A_k)(A_{k+1} \dots A_j)$  dla pewnego  $k \in \{i, \dots, j-1\}$ . Stąd mamy zależność:

$$m(i, j) = \min_{i \leq k < j} \{m(i, k) + m(k+1, j) + p_{i-1} \cdot p_k \cdot p_j\} \quad \text{dla } i < j$$

*Dygresja.* Składnik  $p_{i-1} \cdot p_k \cdot p_j$  to koszt *ostatniego* mnożenia, czyli pomnożenia macierzy będącej wynikiem  $A_i \dots A_k$  przez macierz będącą wynikiem  $A_{k+1} \dots A_j$ . Rozmiar iloczynu ciągu macierzy odczytujemy „teleskopowo” – wewnętrzne wymiary (liczba kolumn poprzedniej macierzy = liczba wierszy następnej) skracają się, a zostają jedynie skrajne:

$$\begin{aligned} A_i \dots A_k : M_{p_{i-1}}^{p_i} \cdot M_{p_i}^{p_{i+1}} \dots M_{p_{k-1}}^{p_k} &\longrightarrow M_{p_{i-1}}^{p_k}, \\ A_{k+1} \dots A_j : M_{p_k}^{p_{k+1}} \cdot M_{p_{k+1}}^{p_{k+2}} \dots M_{p_{j-1}}^{p_j} &\longrightarrow M_{p_k}^{p_j}. \end{aligned}$$

Pozostaje pomnożyć macierz  $M_{p_{i-1}}^{p_k}$  przez macierz  $M_{p_k}^{p_j}$ , a to – zgodnie ze wzorem na koszt mnożenia dwóch macierzy – kosztuje

$$\underbrace{p_{i-1} \cdot p_k \cdot p_j}_{\text{wspólny wymiar}}$$

mnożeń skalarnych.

Niech  $s(i, j)$ , dla  $1 \leq i < j \leq n$ , będzie wartością  $k$ , dla którego to minimum jest osiągane (czyli  $s(i, j)$  wskazuje miejsce ostatniego mnożenia w optymalnym rozwiązaniu podproblemu  $A_i \dots A_j$ ).

Jeśli zaimplementujemy po prostu tę formułę rekurencyjną otrzymamy alg. wykładniczy. Ale jest tylko  $n + \binom{n}{2} = \mathcal{O}(n^2)$  podproblemów, więc rzędu  $\mathcal{O}(n^2)$  liczb  $m(i, j)$  i każdą z tych liczb wystarczy obliczyć jeden raz.

Pomysł programowania dynamicznego polega więc na zapamiętaniu tych wartości w tablicy zamiast liczenia ich wielokrotnie. Wprowadzamy dwie tablice indeksowane parą  $(i, j)$ ,  $1 \leq i \leq j \leq n$  (a ponieważ rozważamy tylko  $i \leq j$ , wypełniona zostaje jedynie górna połowa każdej z nich – jak w przykładzie poniżej):

- $m(i, j)$  – **koszt**: minimalna liczba mnożeń skalarnych dla podproblemu  $A_i \dots A_j$ . Ostateczną odpowiedzią jest  $m(1, n)$ .
- $s(i, j)$  – **cięcie**: miejsce  $k$  ostatniego mnożenia, na którym ten minimalny koszt jest osiągany. Sama tablica  $m$  podaje tylko *ile* mnożeń wystarczy, ale nie *jak* ustawić nawiasy; to  $s$  pozwala odtworzyć optymalne nawiasowanie (wykorzysta ją algorytm MATRIXCHAIN-MULTIPLY dalej).

#### Algorytm 7: Algorytm Matrix Chain Order

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>function</b> MATRIXCHAINORDER( $p$ )<br>1: $n \leftarrow \text{length}(p) - 1$<br>2: <b>for</b> $i \leftarrow 1$ <b>to</b> $n$ <b>do</b><br>3: $m(i, i) \leftarrow 0$<br>4: <b>for</b> $l \leftarrow 2$ <b>to</b> $n$ <b>do</b><br>5: <b>for</b> $i \leftarrow 1$ <b>to</b> $n - l + 1$ <b>do</b><br>6: $j \leftarrow i + l - 1$<br>7: $m(i, j) \leftarrow \infty$<br>8: <b>for</b> $k \leftarrow i$ <b>to</b> $j - 1$ <b>do</b><br>9: $q \leftarrow m(i, k) + m(k + 1, j) + p_{i-1} \cdot p_k \cdot p_j$ | $\triangleright p = (p_0, p_1, \dots, p_n)$ – ciąg rozmiarów<br>$\triangleright A_i$ – macierz $p_{i-1} \times p_i$<br><br>$\triangleright l$ – liczba macierzy w podproblemie<br>$\triangleright l = 2$ : $A_1 A_2, A_2 A_3, \dots, A_{n-1} A_n$<br>$\triangleright l = 3$ : $A_1 A_2 A_3, A_2 A_3 A_4, \dots, A_{n-2} A_{n-1} A_n$<br>$\triangleright l = n$ : $A_1 \dots A_n$ |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

<sup>1</sup> $M_a^b$  – przestrzeń liniowa macierzy o  $a$  wierszach i  $b$  kolumnach (wymiaru  $a \times b$ ).

```

10:         if  $q < m(i, j)$  then
11:              $m(i, j) \leftarrow q$ 
12:              $s(i, j) \leftarrow k$ 
13:     return  $m, s$ 

```

*Dygresja.* Algorytm wypełnia tablicę  $m$  od dołu: zaczyna od najmniejszych podproblemów i rośnie. Kluczowa jest kolejność, w jakiej wypełnimy komórki – tak dobrana, by w chwili liczenia  $m(i, j)$  wszystkie potrzebne mniejsze wartości były już policzone.

- **Pierwsza pętla (po  $i$ )** ustawia przekątną  $m(i, i) = 0$ : pojedyncza macierz nie wymaga żadnego mnożenia.
- **Zmienna  $l$**  to długość podproblemu, czyli liczba macierzy w ciągu  $A_i \dots A_j$ . Wzór  $m(i, j) = \min_k \{ m(i, k) + m(k+1, j) + \dots \}$  odwołuje się wyłącznie do podciągów *krótszych* niż  $A_i \dots A_j$  (bo  $k < j$  oraz  $k+1 > i$ ). Dlatego idziemy po  $l = 2, 3, \dots, n$ : gdy liczymy ciągi długości  $l$ , wszystkie ciągi długości  $< l$  są już w tablicy.
- **Przy ustalonym  $l$**  przesuwamy okno długości  $l$  wzdłuż ciągu:  $i$  to początek okna, a koniec wyznacza  $j = i + l - 1$ . Górna granica  $i \leq n - l + 1$  pilnuje, by okno nie wyszło poza  $A_n$  (komentarze obok pętli pokazują, jak dla kolejnych  $l$  przesuwa się okno).
- **Wewnętrzna pętla po  $k$**  próbuje każdego miejsca ostatniego mnożenia  $A_i \dots A_k \mid A_{k+1} \dots A_j$  i zostawia to o najmniejszym koszcie  $q$ . Przy każdej poprawie zapamiętujemy nie tylko koszt w  $m(i, j)$ , ale i zwycięskie cięcie w  $s(i, j)$  – to ono pozwoli później odtworzyć nawiasowanie.

Innymi słowy:  $l$  mówi *jak duży* podproblem liczymy,  $i$  (a z nim  $j$ ) *który* to podproblem, a  $k$  przeszukuje wszystkie sposoby jego rozcięcia.

Mamy 3 pętle (po  $l, i, k$ ) i liczbę iteracji każdej z tych pętli można ograniczyć przez  $n$ , zatem złożoność czasowa to  $\mathcal{O}(n^3)$ .

### Przykład 3.5 (Optymalna kolejność mnożenia macierzy dla $n = 5$ ).

$p = (4, 5, 2, 1, 2, 3)$ .

$A_1 - 4 \times 5$ ,  $A_2 - 5 \times 2$ ,  $A_3 - 2 \times 1$ ,  $A_4 - 1 \times 2$ ,  $A_5 - 2 \times 3$ .

$$m = \begin{bmatrix} i \backslash j & 1 & 2 & 3 & 4 & 5 \\ 1 & 0 & 40 & 30 & 38 & 48 \\ 2 & \times & 0 & 10 & 20 & 31 \\ 3 & \times & \times & 0 & 4 & 12 \\ 4 & \times & \times & \times & 0 & 6 \\ 5 & \times & \times & \times & \times & 0 \end{bmatrix}$$

$$s = \begin{bmatrix} i \backslash j & 1 & 2 & 3 & 4 & 5 \\ 1 & \times & 1 & 1 & 3 & 3 \\ 2 & \times & \times & 2 & 3 & 3 \\ 3 & \times & \times & \times & 3 & 3 \\ 4 & \times & \times & \times & \times & 4 \\ 5 & \times & \times & \times & \times & \times \end{bmatrix}$$

Prześledźmy, jak rosną obie tablice. Algorytm wypełnia je *przekątnymi*: najpierw  $L = 1$  (przekątna główna), potem  $L = 2$ ,  $L = 3$ , .... Komórki dopisane w danym kroku wyróżniono na **pomarańczowo**; puste pola to podproblemy jeszcze nieobliczone, a  $\times$  – pola nieużywane ( $i \geq j$  dla  $s$ ,  $i > j$  dla  $m$ ).

Stan początkowy ( $L = 1$ , pojedyncze macierze, koszt 0):

$$m = \begin{bmatrix} i \backslash j & 1 & 2 & 3 & 4 & 5 \\ 1 & 0 & & & & \\ 2 & \times & 0 & & & \\ 3 & \times & \times & 0 & & \\ 4 & \times & \times & \times & 0 & \\ 5 & \times & \times & \times & \times & 0 \end{bmatrix}$$

$$s = \begin{bmatrix} i \backslash j & 1 & 2 & 3 & 4 & 5 \\ 1 & \times & & & & \\ 2 & \times & \times & & & \\ 3 & \times & \times & \times & & \\ 4 & \times & \times & \times & \times & \\ 5 & \times & \times & \times & \times & \times \end{bmatrix}$$

---

|       |   |   |   |   |   |   |
|-------|---|---|---|---|---|---|
| $i$   | 0 | 1 | 2 | 3 | 4 | 5 |
| $p_i$ | 4 | 5 | 2 | 1 | 2 | 3 |

 $A_i = p_{i-1} \times p_i$ 

$$\begin{aligned}
L = 2 : \quad m(1,2) &= 4 \cdot 5 \cdot 2 = 40 \\
m(2,3) &= 5 \cdot 2 \cdot 1 = 10 \\
m(3,4) &= 2 \cdot 1 \cdot 2 = 4 \\
m(4,5) &= 1 \cdot 2 \cdot 3 = 6
\end{aligned}$$

po  $L = 2$  (pary sąsiednich macierzy):

$$m \left[ \begin{array}{c|c|c|c|c|c} i \setminus j & 1 & 2 & 3 & 4 & 5 \\ \hline 1 & 0 & 40 & & & \\ \hline 2 & \times & 0 & 10 & & \\ \hline 3 & \times & \times & 0 & 4 & \\ \hline 4 & \times & \times & \times & 0 & 6 \\ \hline 5 & \times & \times & \times & \times & 0 \end{array} \right]$$

$$s \left[ \begin{array}{c|c|c|c|c|c} i \setminus j & 1 & 2 & 3 & 4 & 5 \\ \hline 1 & \times & 1 & & & \\ \hline 2 & \times & \times & 2 & & \\ \hline 3 & \times & \times & \times & 3 & \\ \hline 4 & \times & \times & \times & \times & 4 \\ \hline 5 & \times & \times & \times & \times & \times \end{array} \right]$$


---

|       |   |   |   |   |   |   |
|-------|---|---|---|---|---|---|
| $i$   | 0 | 1 | 2 | 3 | 4 | 5 |
| $p_i$ | 4 | 5 | 2 | 1 | 2 | 3 |

 $A_i = p_{i-1} \times p_i$ 

$$\begin{aligned}
L = 3 : \quad m(1,3) &= \min \begin{cases} m(1,1) + m(2,3) + 4 \cdot 5 \cdot 1 \\ m(1,2) + m(3,3) + 4 \cdot 2 \cdot 1 \end{cases} = \min \begin{cases} 0 + 10 + 20 \\ 40 + 0 + 8 \end{cases} = \min \begin{cases} 30 \\ 48 \end{cases} = 30 \\
m(2,4) &= \min \begin{cases} m(2,2) + m(3,4) + 5 \cdot 2 \cdot 2 \\ m(2,3) + m(4,4) + 5 \cdot 1 \cdot 2 \end{cases} = \min \begin{cases} 0 + 4 + 20 \\ 10 + 0 + 10 \end{cases} = \min \begin{cases} 24 \\ 20 \end{cases} = 20 \\
m(3,5) &= \min \begin{cases} m(3,3) + m(4,5) + 2 \cdot 1 \cdot 3 \\ m(3,4) + m(5,5) + 2 \cdot 2 \cdot 3 \end{cases} = \min \begin{cases} 0 + 6 + 6 \\ 4 + 0 + 12 \end{cases} = \min \begin{cases} 12 \\ 16 \end{cases} = 12
\end{aligned}$$

po  $L = 3$ :

$$m \left[ \begin{array}{c|c|c|c|c|c} i \setminus j & 1 & 2 & 3 & 4 & 5 \\ \hline 1 & 0 & 40 & 30 & & \\ \hline 2 & \times & 0 & 10 & 20 & \\ \hline 3 & \times & \times & 0 & 4 & 12 \\ \hline 4 & \times & \times & \times & 0 & 6 \\ \hline 5 & \times & \times & \times & \times & 0 \end{array} \right]$$

$$s \left[ \begin{array}{c|c|c|c|c|c} i \setminus j & 1 & 2 & 3 & 4 & 5 \\ \hline 1 & \times & 1 & 1 & & \\ \hline 2 & \times & \times & 2 & 3 & \\ \hline 3 & \times & \times & \times & 3 & 3 \\ \hline 4 & \times & \times & \times & \times & 4 \\ \hline 5 & \times & \times & \times & \times & \times \end{array} \right]$$


---

|       |   |   |   |   |   |   |
|-------|---|---|---|---|---|---|
| $i$   | 0 | 1 | 2 | 3 | 4 | 5 |
| $p_i$ | 4 | 5 | 2 | 1 | 2 | 3 |

 $A_i = p_{i-1} \times p_i$ 

$$\begin{aligned}
L = 4 : \quad m(1,4) &= \min \begin{cases} m(1,1) + m(2,4) + 4 \cdot 5 \cdot 2 \\ m(1,2) + m(3,4) + 4 \cdot 2 \cdot 2 \\ m(1,3) + m(4,4) + 4 \cdot 1 \cdot 2 \end{cases} = \min \begin{cases} 0 + 20 + 40 \\ 40 + 4 + 16 \\ 30 + 0 + 8 \end{cases} = \min \begin{cases} 60 \\ 60 \\ 38 \end{cases} = 38 \\
m(2,5) &= \min \begin{cases} m(2,2) + m(3,5) + 5 \cdot 2 \cdot 3 \\ m(2,3) + m(4,5) + 5 \cdot 1 \cdot 3 \\ m(2,4) + m(5,5) + 5 \cdot 2 \cdot 3 \end{cases} = \min \begin{cases} 0 + 12 + 30 \\ 10 + 6 + 15 \\ 20 + 0 + 30 \end{cases} = \min \begin{cases} 42 \\ 31 \\ 50 \end{cases} = 31
\end{aligned}$$

po  $L = 4$ :

|     |                  |   |    |    |    |    |
|-----|------------------|---|----|----|----|----|
| $m$ | $i \backslash j$ | 1 | 2  | 3  | 4  | 5  |
|     | 1                | 0 | 40 | 30 | 38 |    |
|     | 2                | × | 0  | 10 | 20 | 31 |
|     | 3                | × | ×  | 0  | 4  | 12 |
|     | 4                | × | ×  | ×  | 0  | 6  |
|     | 5                | × | ×  | ×  | ×  | 0  |

|     |                  |   |   |   |   |   |
|-----|------------------|---|---|---|---|---|
| $s$ | $i \backslash j$ | 1 | 2 | 3 | 4 | 5 |
|     | 1                | × | 1 | 1 | 3 |   |
|     | 2                | × | × | 2 | 3 | 3 |
|     | 3                | × | × | × | 3 | 3 |
|     | 4                | × | × | × | × | 4 |
|     | 5                | × | × | × | × | × |

|       |   |   |   |   |   |   |
|-------|---|---|---|---|---|---|
| $i$   | 0 | 1 | 2 | 3 | 4 | 5 |
| $p_i$ | 4 | 5 | 2 | 1 | 2 | 3 |

$A_i = p_{i-1} \times p_i$

$$L = 5 : \quad m(1, 5) = \min \begin{cases} m(1, 1) + m(2, 5) + 4 \cdot 5 \cdot 3 \\ m(1, 2) + m(3, 5) + 4 \cdot 2 \cdot 5 \\ m(1, 3) + m(4, 5) + 4 \cdot 1 \cdot 3 \\ m(1, 4) + m(5, 5) + 4 \cdot 2 \cdot 3 \end{cases} = \min \begin{cases} 0 + 31 + 60 \\ 40 + 12 + 24 \\ 30 + 6 + 12 \\ 38 + 0 + 24 \end{cases} = \min \begin{cases} 91 \\ 76 \\ 48 \\ 62 \end{cases} = 48$$

po  $L = 5$  (cały ciąg – ostatnia komórka):

|     |                  |   |    |    |    |    |
|-----|------------------|---|----|----|----|----|
| $m$ | $i \backslash j$ | 1 | 2  | 3  | 4  | 5  |
|     | 1                | 0 | 40 | 30 | 38 | 48 |
|     | 2                | × | 0  | 10 | 20 | 31 |
|     | 3                | × | ×  | 0  | 4  | 12 |
|     | 4                | × | ×  | ×  | 0  | 6  |
|     | 5                | × | ×  | ×  | ×  | 0  |

|     |                  |   |   |   |   |   |
|-----|------------------|---|---|---|---|---|
| $s$ | $i \backslash j$ | 1 | 2 | 3 | 4 | 5 |
|     | 1                | × | 1 | 1 | 3 | 3 |
|     | 2                | × | × | 2 | 3 | 3 |
|     | 3                | × | × | × | 3 | 3 |
|     | 4                | × | × | × | × | 4 |
|     | 5                | × | × | × | × | × |

Każdy przebieg pętli po  $l$  dopisuje dokładnie jedną przekątną: w komórce  $m(i, j)$  ląduje koszt, a w bliźniaczej  $s(i, j)$  – zwycięskie cięcie  $k$ . Ostatnią obliczoną wartością jest  $m(1, 5) = 48$  w prawym górnym rogu – szukany koszt całego ciągu.

Oto algorytm znajdowania optymalnego rozwiązania:

#### Algorytm 8: Algorytm Matrix Chain Multiply

```

function MATRIXCHAINMULTIPLY( $A, s, i, j$ )                                ▷  $A = (A_1, \dots, A_n)$ 
1:   if  $j = i$  then
2:     return  $A_i$ 
3:    $X \leftarrow$  MATRIXCHAINMULTIPLY( $A, s, i, s(i, j)$ )
4:    $Y \leftarrow$  MATRIXCHAINMULTIPLY( $A, s, s(i, j) + 1, j$ )
5:   return MATRIXMULTIPLY( $X, Y$ )
    
```

W celu znalezienia rozwiązania wywołujemy:  $\text{MATRIXCHAINMULTIPLY}(A, s, 1, n)$

$$A_1 \cdot A_2 \cdot A_3 \cdot A_4 \cdot A_5$$

$$s(1, 5) = 3 \implies (A_1 \cdot A_2 \cdot A_3)(A_4 \cdot A_5)$$

$$s(1, 3) = 1 \implies (A_1 \cdot (A_2 \cdot A_3))(A_4 \cdot A_5) \quad s(4, 5) = 4 \implies \text{bez zmian}$$

$$s(2, 3) = 2 \implies (A_1 \cdot (A_2 \cdot A_3))(A_4 \cdot A_5)$$

Aby algorytm zadziałał dobrze, problem musi spełniać warunki:

1. Własność optymalnej podstruktury – optymalne rozwiązanie problemu można skonstruować z optymalnych rozwiązań podproblemów.
2. Własność wspólnych podproblemów – sytuacja, gdy prosty algorytm rekurencyjny obliczałby rozwiązania tych samych podproblemów wielokrotnie. Liczba podproblemów powinna być „mała”.

W klasycznej metodzie programowania dynamicznego rozwiązanie jest budowane od dołu do góry – od podproblemów najmniejszych do największych. Jest też podejście, w którym rozwiązanie jest budowane od góry do dołu, podobnie jak w prostym algorytmie rekurencyjnym, ale z tą różnicą, że unikamy obliczania tych samych podproblemów wielokrotnie. To podejście nazywa się spamiętywaniem (*memoization*).

**Problem mnożenia ciągu macierzy ze spamiętywaniem** Algorytm oblicza tablice  $m$  i  $s$  jak poprzednio, ale w inny sposób. Dopóki podproblem nie jest rozwiązany,  $m(i, j)$  przechowuje  $\infty$ . Po rozwiązaniu podproblemu  $(i, j)$  umieszczamy w  $m(i, j)$  odpowiednią wartość i od tej pory używamy jej bez obliczania.

#### Algorytm 9: Algorytm Lookup Chain

```

function LOOKUPCHAIN( $p, i, j$ )      ▷  $p = (p_0, \dots, p_n)$  - ciąg wymiarów;  $A_i$  - macierz  $p_{i-1} \times p_i$ 
1:  if  $m(i, j) < \infty$  then
2:    return  $m(i, j), s(i, j)$ 
3:  if  $i = j$  then
4:     $m(i, j) \leftarrow 0$ 
5:  else
6:    for  $k \leftarrow i$  to  $j - 1$  do
7:       $q \leftarrow \text{LOOKUPCHAIN}(p, i, k) + \text{LOOKUPCHAIN}(p, k + 1, j) + p_{i-1} \cdot p_k \cdot p_j$ 
8:      if  $q < m(i, j)$  then
9:         $m(i, j) \leftarrow q$ 
10:        $s(i, j) \leftarrow k$ 
11:  return  $(m(i, j), s(i, j))$ 

```

#### Algorytm 10: Algorytm Memoized Matrix Chain

```

function MEMOIZEDMATRIXCHAIN( $p$ )
1:   $n \leftarrow \text{length}(p) - 1$ 
2:  for  $i \leftarrow 1$  to  $n$  do
3:    for  $j \leftarrow i$  to  $n$  do
4:       $m(i, j) \leftarrow \infty$ 
5:  return LOOKUPCHAIN( $p, 1, n$ )

```

#### Złożoność czasowa:

- Linie 1–4 w MEMOIZEDMATRIXCHAIN zajmują  $\Theta(n^2)$ .
- Każdy z  $\mathcal{O}(n^2)$  elementów tablicy  $m$  jest modyfikowany raz, później odczytanie wartości  $m(i, j)$  zajmuje czas stały.
- Modyfikacja  $m(i, j)$  zajmuje  $\mathcal{O}(n)$  czasu (pętla w liniach 5–9 w LOOKUPCHAIN).

Ostatecznie złożoność algorytmu wynosi  $\mathcal{O}(n^3)$ .

### Problem najdłuższego wspólnego podciągu

Niech  $X = (x_1, \dots, x_m)$  – ciąg.

$Z = (z_1, \dots, z_k)$  jest podciągiem  $X$ , jeśli istnieje rosnący ciąg indeksów  $1 \leq i_1 < i_2 < \dots < i_k \leq m$  takich, że dla każdego  $j \in \{1, \dots, k\}$ ,  $z_j = x_{i_j}$ .

#### Przykład 3.6 (Podciąg).

$X = (A, B, C, B, D, A, B)$ ,  $Z = (B, C, D, B)$ . Indeksy: (2, 3, 5, 7).

#### Problem 3.7 (najdłuższego wspólnego podciągu).

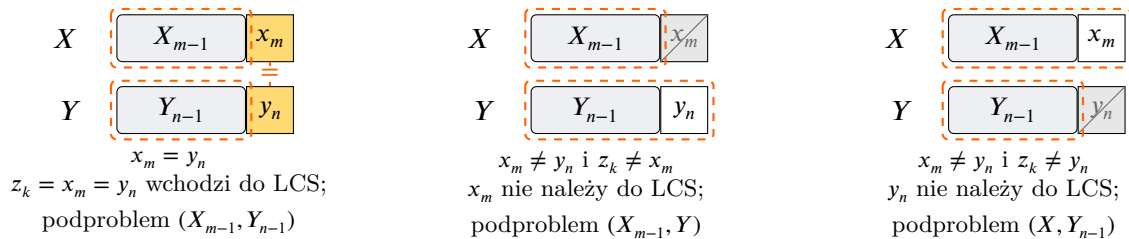
Dane są ciągi  $X = (x_1, \dots, x_m)$  oraz  $Y = (y_1, \dots, y_n)$ . Należy znaleźć najdłuższy wspólny podciąg (LCS – *Longest Common Subsequence*)  $X$  i  $Y$ , czyli najdłuższy ciąg, który jest podciągiem zarówno  $X$ , jak i  $Y$ .

Wszystkich podciągów  $X$  jest  $2^m$ , więc algorytm sprawdzający wszystkie podciągi  $X$  działa w czasie wykładniczym.

Niech  $X_i = (x_1, \dots, x_i)$  dla  $i \in \{1, \dots, m\}$  i dodatkowo  $X_0 = ()$  (ciąg pusty). Analogicznie dla  $Y$ .

**Lemat 3.8.** Niech  $X = (x_1, \dots, x_m)$ ,  $Y = (y_1, \dots, y_n)$  i niech  $Z = (z_1, \dots, z_k)$  będzie LCS  $X$  i  $Y$ .

1. Jeśli  $x_m = y_n$ , to  $z_k = x_m = y_n$  i  $Z_{k-1}$  jest LCS  $X_{m-1}$  i  $Y_{n-1}$ .
2. Jeśli  $x_m \neq y_n$  i  $z_k \neq x_m$ , to  $Z$  jest LCS  $X_{m-1}$  i  $Y$ .
3. Jeśli  $x_m \neq y_n$  i  $z_k \neq y_n$ , to  $Z$  jest LCS  $X$  i  $Y_{n-1}$ .



Rysunek 5: Trzy przypadki lematu o optymalnej podstrukturze LCS. Żółta komórka – dopasowany ostatni element ( $x_m = y_n$ ), który wchodzi do LCS; szara, przekreślona komórka – element odrzucony, nienależący do tego LCS; przerywana pomarańczowa ramka – podproblem(y), do których redukuje się zadanie.

*Dowód.*

1. Załóżmy, że  $x_m = y_n$ . Gdyby  $z_k \neq x_m$ , to moglibyśmy wydłużyć podciąg  $Z$  przez dodanie  $z_{k+1} = x_m = y_n$ , co daje sprzeczność, bo  $(z_1, \dots, z_{k+1})$  jest wspólnym podciągiem  $X$  i  $Y$  dłuższym niż  $Z$ . Zatem  $z_k = x_m = y_n$  i  $Z_{k-1}$  jest wspólnym podciągiem  $X_{m-1}$  i  $Y_{n-1}$ . Gdyby istniał wspólny podciąg  $W$  ciągów  $X_{m-1}$  i  $Y_{n-1}$  długości  $> k - 1$ , to dodając na koniec  $W$  element  $x_m = y_n$  otrzymalibyśmy wspólny podciąg  $X$  i  $Y$  długości  $> k - 1 + 1 = k$ . Sprzeczność, bo długość  $Z$  wynosi  $k$ .
2. Załóżmy, że  $x_m \neq y_n$  i  $z_k \neq x_m$ . Wtedy  $Z$  jest wspólnym podciągiem ciągów  $X_{m-1}$  i  $Y$ . Gdyby istniał wspólny podciąg  $W$  ciągów  $X_{m-1}$  i  $Y$  o długości  $> k$ , to  $W$  byłby wspólnym podciągiem ciągów  $X$  i  $Y$  dłuższym niż  $Z$ . Sprzeczność.
3. Symetrycznie do punktu 2.

□

Rozwiązanie dla problemu  $(X, Y)$  można skonstruować z rozwiązań podproblemów  $(X_{m-1}, Y_{n-1})$ ,  $(X_{m-1}, Y)$  i  $(X, Y_{n-1})$  – **własność optymalnej podstruktury**. Podproblem  $(X_{m-1}, Y_{n-1})$  jest zawarty w podproblemach  $(X_{m-1}, Y)$  i  $(X, Y_{n-1})$  – **własność wspólnych podproblemów**.

Niech  $c(i, j)$  oznacza długość LCS ciągów  $X_i$  i  $Y_j$ .

$$c(i, j) = \begin{cases} 0 & \text{jeśli } i = 0 \text{ lub } j = 0 \\ c(i-1, j-1) + 1 & \text{jeśli } i, j > 0 \text{ i } x_i = y_j \\ \max\{c(i-1, j), c(i, j-1)\} & \text{jeśli } i, j > 0 \text{ i } x_i \neq y_j \end{cases}$$

Będziemy też używać tablicy  $b(i, j)$ , aby znaleźć sam podciąg LCS. Tablice mają wymiary:  $c[0 \dots m, 0 \dots n]$  oraz  $b[1 \dots m, 1 \dots n]$ . Wartości w tablicy kierunków to  $b(i, j) \in \{\nwarrow, \uparrow, \leftarrow\}$ .

#### Algorytm 11: Algorytm obliczający długość LCS

```

function LCS( $X, Y$ )
1:   $m \leftarrow \text{length}(X)$ 
2:   $n \leftarrow \text{length}(Y)$ 
3:  for  $i \leftarrow 0$  to  $m$  do
4:     $c(i, 0) \leftarrow 0$ 
5:  for  $j \leftarrow 0$  to  $n$  do
6:     $c(0, j) \leftarrow 0$ 
7:  for  $i \leftarrow 1$  to  $m$  do
8:    for  $j \leftarrow 1$  to  $n$  do
9:      if  $x_i = y_j$  then
10:          $c(i, j) \leftarrow c(i-1, j-1) + 1$ 
11:          $b(i, j) \leftarrow \text{"}\nwarrow\text{"}$ 
12:      else if  $c(i-1, j) \geq c(i, j-1)$  then
13:          $c(i, j) \leftarrow c(i-1, j)$ 
14:          $b(i, j) \leftarrow \text{"}\uparrow\text{"}$ 
15:      else
16:          $c(i, j) \leftarrow c(i, j-1)$ 
17:          $b(i, j) \leftarrow \text{"}\leftarrow\text{"}$ 
18:  return  $(c, b)$ 
    
```

**Przykład:**  $X = (A, B, C, B, D, A, B)$ ,  $Y = (B, D, C, A, B, A)$ .  
 $m = 7$ ,  $n = 6$ .

Tablica kosztów  $c$ :

|     | $j$   | 0   | 1   | 2   | 3   | 4   | 5   | 6 |
|-----|-------|-----|-----|-----|-----|-----|-----|---|
| $i$ | $y_j$ | $B$ | $D$ | $C$ | $A$ | $B$ | $A$ |   |
| 0   | $x_i$ | 0   | 0   | 0   | 0   | 0   | 0   | 0 |
| 1   | $A$   | 0   | 0   | 0   | 0   | 1   | 1   | 1 |
| 2   | $B$   | 0   | 1   | 1   | 1   | 1   | 2   | 2 |
| 3   | $C$   | 0   | 1   | 1   | 2   | 2   | 2   | 2 |
| 4   | $B$   | 0   | 1   | 1   | 2   | 2   | 3   | 3 |
| 5   | $D$   | 0   | 1   | 2   | 2   | 2   | 3   | 3 |
| 6   | $A$   | 0   | 1   | 2   | 2   | 3   | 3   | 4 |
| 7   | $B$   | 0   | 1   | 2   | 2   | 3   | 4   | 4 |

Tablica strzałek  $b$ :

|     | $j$   | 0   | 1          | 2            | 3            | 4            | 5            | 6            |
|-----|-------|-----|------------|--------------|--------------|--------------|--------------|--------------|
| $i$ | $y_j$ | $B$ | $D$        | $C$          | $A$          | $B$          | $A$          |              |
| 0   | $x_i$ |     |            |              |              |              |              |              |
| 1   | $A$   |     | $\uparrow$ | $\uparrow$   | $\uparrow$   | $\nwarrow$   | $\leftarrow$ | $\nwarrow$   |
| 2   | $B$   |     | $\nwarrow$ | $\leftarrow$ | $\leftarrow$ | $\uparrow$   | $\nwarrow$   | $\leftarrow$ |
| 3   | $C$   |     | $\uparrow$ | $\uparrow$   | $\nwarrow$   | $\leftarrow$ | $\uparrow$   | $\uparrow$   |
| 4   | $B$   |     | $\nwarrow$ | $\uparrow$   | $\uparrow$   | $\uparrow$   | $\nwarrow$   | $\leftarrow$ |
| 5   | $D$   |     | $\uparrow$ | $\nwarrow$   | $\uparrow$   | $\uparrow$   | $\uparrow$   | $\uparrow$   |
| 6   | $A$   |     | $\uparrow$ | $\uparrow$   | $\uparrow$   | $\nwarrow$   | $\uparrow$   | $\nwarrow$   |
| 7   | $B$   |     | $\nwarrow$ | $\uparrow$   | $\uparrow$   | $\uparrow$   | $\nwarrow$   | $\uparrow$   |

- **Dopasowanie** ( $\searrow$ ). Komórka  $c(2, 1)$ :  $x_2 = B$ ,  $y_1 = B$ , więc  $x_i = y_j$ . Bierzymy wartość z przekątnej i dodajemy 1:  $c(2, 1) = c(1, 0) + 1 = 0 + 1 = 1$ . Ostatnie znaki się zgadzają, więc element wchodzi do LCS i strzałka wskazuje  $\searrow$ .
- **Brak dopasowania, w górę** ( $\uparrow$ ). Komórka  $c(3, 1)$ :  $x_3 = C$ ,  $y_1 = B$ , więc  $x_i \neq y_j$ . Porównujemy sąsiadów:  $c(2, 1) = 1$  (góra) oraz  $c(3, 0) = 0$  (lewo). Ponieważ  $c(i - 1, j) \geq c(i, j - 1)$ , bierzemy  $c(3, 1) = c(2, 1) = 1$  i strzałkę  $\uparrow$ .
- **Brak dopasowania, w lewo** ( $\leftarrow$ ). Komórka  $c(2, 2)$ :  $x_2 = B$ ,  $y_2 = D$ , więc  $x_i \neq y_j$ . Sąsiedzi:  $c(1, 2) = 0$  (góra) oraz  $c(2, 1) = 1$  (lewo). Tym razem  $c(i - 1, j) < c(i, j - 1)$ , więc  $c(2, 2) = c(2, 1) = 1$  i strzałka  $\leftarrow$ .
- **Remis**. Komórka  $c(1, 1)$ :  $x_1 = A \neq B = y_1$ , a obaj sąsiedzi są równi:  $c(0, 1) = c(1, 0) = 0$ . Warunek  $c(i - 1, j) \geq c(i, j - 1)$  jest spełniony (równość), więc algorytm wybiera  $\uparrow$  – remis rozstrzygamy zawsze na korzyść góry.

```

1:   if  $i = 0$  lub  $j = 0$  then
2:       return  $\varepsilon$ 
3:   if  $b(i, j) = „\nwarrow”$  then
4:       PRINTLCS( $b, X, Y, i - 1, j - 1$ )
5:       print  $x_i$ 
6:   else if  $b(i, j) = „\nearrow”$  then
7:       PRINTLCS( $b, X, Y, i - 1, j$ )
8:   else
9:       PRINTLCS( $b, X, Y, i, j - 1$ )

```

*Dygresja.* Prześledźmy to wywołanie. Zaczynamy od prawego dolnego rogu i podążamy za strzałkami z tablicy  $b$  aż do brzegu tablicy. Przy  $\nwarrow$  schodzimy po przekątnej do  $(i-1, j-1)$  i zaznaczamy znak  $x_i$  jako należący do LCS; przy  $\uparrow$  idziemy do  $(i-1, j)$ , a przy  $\leftarrow$  do  $(i, j-1)$  (bez wypisywania).

$$\begin{array}{ccc} (7,6) & \xrightarrow{\uparrow} (6,6) & \xrightarrow{\nearrow} (5,5) \quad [x_6 = A] \\ & \xrightarrow{\uparrow} (4,5) & \xrightarrow{\nearrow} (3,4) \quad [x_4 = B] \\ & \xleftarrow{\leftarrow} (3,3) & \xleftarrow{\nwarrow} (2,2) \quad [x_3 = C] \\ & \xleftarrow{\leftarrow} (2,1) & \xleftarrow{\nwarrow} (1,0) \quad [x_2 = B] \end{array}$$

Złożoność czasowa tego algorytmu wynosi  $\Theta(m \cdot n)$ .

## Wykład 6

### Algorytmy Zachłanne

**Pomysł:** Algorytm w każdym kroku dokonuje lokalnie najlepszego wyboru (najlepszego w danym momencie).

#### Problem 4.1 (wyboru zajęć).

$S = \{z_1, \dots, z_n\}$  – zbiór zajęć wymagający tych samych zasobów.

$s_i$  – czas rozpoczęcia zajęcia  $z_i$

$s_i, t_i \in \mathbb{N}, \quad s_i < t_i$

$t_i$  – czas zakończenia zajęcia  $z_i$

Zajęcia  $z_i$  i  $z_j$  nazywamy zgodnymi, jeśli  $\langle s_i, t_i \rangle \cap \langle s_j, t_j \rangle = \emptyset$ . Zadanie polega na znalezieniu podzbioru o największej liczności  $A \subseteq S$  parami zgodnych zajęć.

**Przykład 4.2.** Przykładowy zbiór zajęć:

|       |   |   |   |    |   |   |
|-------|---|---|---|----|---|---|
| $i$   | 1 | 2 | 3 | 4  | 5 | 6 |
| $s_i$ | 0 | 2 | 4 | 7  | 1 | 7 |
| $t_i$ | 4 | 6 | 7 | 10 | 3 | 9 |

Porządkujemy zajęcia w czasie niemalejącym względem czasu zakończenia. To zajmuje  $\mathcal{O}(n \log n)$  czasu. Od tej pory zakładamy  $t_1 \leq t_2 \leq \dots \leq t_n$  (zajęcia są już posortowane).

**Przykład 4.3.** Zajęcia z przykładu 4.2 po posortowaniu względem czasu zakończenia:

|       |   |   |   |   |   |    |
|-------|---|---|---|---|---|----|
| $i$   | 1 | 2 | 3 | 4 | 5 | 6  |
| $s_i$ | 1 | 0 | 2 | 4 | 7 | 7  |
| $t_i$ | 3 | 4 | 6 | 7 | 9 | 10 |

**Podejście programowania dynamicznego** Definiujemy zbiory zajęć:

$$S_{ij} = \{z_k \in S : t_i \leq s_k < t_k \leq s_j\}$$

(zbiór zajęć zaczynających się po czasie zakończenia zajęcia  $z_i$  i kończących się przed czasem rozpoczęcia zajęcia  $z_j$ ).

Dodajemy 2 sztuczne zajęcia:  $z_0$  z czasem  $t_0 = 0$  i  $z_{n+1}$  z czasem  $s_{n+1} = \infty$ . Wtedy  $S = S_{0,n+1}$ . Jeśli  $i \geq j$ , to  $S_{ij} = \emptyset$ , bo  $s_j < t_j \leq t_i$  (bo zajęcia są posortowane).

*Dygresja.* Pokażmy dokładniej, dlaczego  $S_{ij} = \emptyset$  dla  $i \geq j$ . Przypuśćmy, że istnieje  $z_k \in S_{ij}$ . Z definicji  $S_{ij}$  mamy wtedy  $t_i \leq s_k < t_k \leq s_j$ , a ponieważ zajęcia są posortowane i  $i \geq j$ , to  $t_j \leq t_i$ , skąd  $s_j < t_j \leq t_i$ . Łącząc obie nierówności, otrzymujemy  $s_j < t_i \leq s_k < t_k \leq s_j$ , czyli  $s_j < s_j$  – sprzeczność.

Problem: znaleźć podzbiór o największej liczności zbioru  $S_{ij}$  składający się z parami zgodnych zajęć dla  $0 \leq i \leq j \leq n+1$ .

Niech  $A_{ij}$  będzie jakimkolwiek podzbiorem największej liczności składającym się z parami zgodnych zajęć zbioru  $S_{ij}$  oraz niech  $c(i, j) = |A_{ij}|$ .

**Podproblem  $S_{ij}$ :** Załóżmy, że  $z_k \in S_{ij}$ . Jeśli wybierzemy  $z_k$  do zbioru rozwiązania, to otrzymamy dwa zbiory (podproblemy):

- $S_{ik}$  (zbiór zajęć, które zaczynają się po zakończeniu zajęcia  $z_i$  i kończą się przed rozpoczęciem zajęcia  $z_k$ ),
- $S_{kj}$  (zbiór zajęć, które zaczynają się po zakończeniu zajęcia  $z_k$  i kończą przed rozpoczęciem zajęcia  $z_j$ ).

Rozwiązanie dla  $S_{ij}$  będzie sumą rozwiązań dla  $S_{ik}$  oraz  $S_{kj}$  oraz elementu  $\{z_k\}$ .

**Własność optymalnej podstruktury:** Załóżmy, że  $z_k \in A_{ij}$ . Wtedy  $A_{ij} = A_{ik} \cup \{z_k\} \cup A_{kj}$ . Gdyby istniało rozwiązanie  $A'_{ik}$  dla  $S_{ik}$  takie, że  $|A'_{ik}| > |A_{ik}|$ , to zbiór  $A'_{ij} = A'_{ik} \cup \{z_k\} \cup A_{kj}$  byłby rozwiązaniem dla  $S_{ij}$  takim, że  $|A'_{ij}| > |A_{ij}|$ . Sprzeczność. Analogicznie dla  $S_{kj}$ . Zatem  $A_{ik}$  oraz  $A_{kj}$  muszą być rozwiązaniami optymalnymi dla odpowiednio  $S_{ik}$  i  $S_{kj}$ .

Stąd wzór rekurencyjny:

$$\begin{cases} c(i, j) = 0 & \text{dla } i \geq j \\ c(i, j) = \max_{i < k < j} \{c(i, k) + c(k, j) + 1\} \end{cases}$$

Używając tego wzoru rekurencyjnego możemy skonstruować algorytm programowania dynamicznego. Mamy  $\mathcal{O}(n^2)$  podproblemów. Dla  $j > i$  mamy  $j - i - 1$  wartości  $k$  do sprawdzenia, czyli rozwiązanie jednego podproblemu zajmie czas liniowy. Otrzymujemy algorytm o złożoności  $\mathcal{O}(n^3)$ .

### Algorytm zachłanny

**Lemat 4.4.** Niech  $z_m \in S_{ij}$  będzie zajęciem takim, że  $t_m = \min\{t_k : z_k \in S_{ij}\}$ . Wtedy:

1. Istnieje podzbiór o największej liczności składający się z parami zgodnych zajęć zbioru  $S_{ij}$  zawierający  $z_m$ .
2.  $S_{im} = \emptyset$  (zatem po wybraniu  $z_m$  do zbioru rozwiązania zostaje co najwyżej 1 niepusty podproblem do rozwiązania –  $S_{mj}$ ).

*Dowód.*

**Ad 1.** Niech  $A_{ij}$  będzie podzbiorem o największej liczności składającym się z parami zgodnych zajęć zbioru  $S_{ij}$ . Niech  $z_k$  będzie zajęciem z  $A_{ij}$  o najmniejszym czasie zakończenia. Jeśli  $z_k = z_m$ , to teza jest udowodniona. Przypuśćmy, że  $z_k \neq z_m$  i rozważmy zbiór  $A'_{ij} = A_{ij} \setminus \{z_k\} \cup \{z_m\}$ . Skoro  $t_m \leq t_k$  oraz zajęcia z  $A_{ij}$  są parami zgodne, to zajęcia z  $A'_{ij}$  również są parami zgodne. Mamy  $|A_{ij}| = |A'_{ij}|$ , więc  $A'_{ij}$  jest rozwiązaniem optymalnym zawierającym  $z_m$ .

**Ad 2.** Przypuśćmy, że nie. Niech  $z_k \in S_{im}$ . Wtedy  $t_i \leq s_k < t_k \leq s_m < t_m \leq s_j$ . Zatem  $z_k \in S_{ij}$  oraz  $t_k < t_m$  – sprzeczność z definicją zajęcia  $z_m$ .  $\square$

Z lematu wynika, że nie musimy sprawdzać  $j - i - 1$  wartości  $k$ , możemy wybrać zajęcie o najmniejszym czasie zakończenia  $z_m \in S_{ij}$ . Żeby rozwiązać podproblem  $S_{ij}$  możemy dodać  $z_m$  do zbioru rozwiązania a następnie dodać do zbioru rozwiązania rozwiązanie optymalne podproblemu  $S_{mj}$ . Zaczynamy z  $S = S_{0,n+1}$  i  $t_0 = 0$ . W pierwszym kroku algorytm wybiera  $z_1$ .

**Algorytm 13:** Zachłanny wybór zajęć

```

function GREEDYACTIVITYSELECTOR( $s, t$ )    ▷ zajęcia są posortowane po czasie zakończeń
1:   $n \leftarrow \text{length}(s)$ 
2:   $A \leftarrow \{z_1\}$                                 ▷  $A$  - zbiór rozwiązań
3:   $j \leftarrow 1$ 
4:  for  $i \leftarrow 2$  to  $n$  do
5:    if  $s_i \geq t_j$  then
6:       $A \leftarrow A \cup \{z_i\}$ 
7:       $j \leftarrow i$ 
8:  return  $A$ 

```

Algorytm zachłanny w każdym kroku wybiera zajęcie zgodne ze wszystkimi zajęciami poprzednio wybranymi, o najmniejszym czasie zakończenia, aby zostawić jak najwięcej miejsca dla zajęć następnych (algorytm zachłanny). Skoro one są posortowane, jeśli  $j < k$  i  $z_k$  jest zgodne z  $z_j$ , to  $z_k$  jest też zgodne z  $z_i$  dla każdego  $1 \leq i < j$ .

**Analiza poprawności:** Algorytm zwraca zbiór parami zgodnych zajęć, bo dodaje do zbioru rozwiązania zajęcie tylko jeśli jest zgodne ze wszystkimi zajęciami dodanymi wcześniej.

Pokażemy przez indukcję po  $n$ , że algorytm znajduje rozwiązanie optymalne.

- Dla  $n = 1$  oczywiste ✓
- Niech  $n > 1$ . Z lematu istnieje optymalne rozwiązanie  $A_1$  takie, że  $z_1 \in A_1$ .  
Rozważmy problem dla  $S' = \{z_i \in S : s_i \geq t_1\}$ . Oczywiście zachodzi  $|S'| < |S|$ .  
Dla każdego  $z_j \in A_1 \setminus \{z_1\}$ , mamy  $z_j \in S'$ , bo  $s_j \geq t_1$ , co wynika z faktu, że  $A_1$  jest rozwiązaniem dla  $S$ .  
Pokażemy, że  $A_1 \setminus \{z_1\}$  jest rozwiązaniem optymalnym dla  $S'$ .  
Przypuśćmy, że istnieje rozwiązanie  $B$  dla  $S'$  takie, że  $|B| > |A_1 \setminus \{z_1\}|$ . Wtedy  $\{z_1\} \cup B$  byłoby rozwiązaniem dla  $S$  takim, że:

$$|\{z_1\} \cup B| = 1 + |B| > 1 + |A_1 \setminus \{z_1\}| = |A_1|$$

Otrzymujemy sprzeczność, bo  $A_1$  jest z założenia rozwiązaniem optymalnym.

Z założenia indukcyjnego nasz algorytm znajduje rozwiązanie optymalne  $B'$  dla  $S'$ . Wtedy  $|B'| = |A_1 \setminus \{z_1\}| = |A_1| - 1$ .

Zbiór  $\{z_1\} \cup B'$  jest rozwiązaniem znalezionym przez algorytm dla  $S$  i:

$$|\{z_1\} \cup B'| = 1 + |B'| = 1 + |A_1| - 1 = |A_1|$$

Więc  $\{z_1\} \cup B'$  jest rozwiązaniem optymalnym.

**Złożoność czasowa**

- linia 1 –  $\mathcal{O}(n)$
- linia 2 –  $\mathcal{O}(1)$
- iteracji pętli w linii 3 jest  $n - 1$
- wewnątrz pętli zajmuje czas  $\mathcal{O}(1)$

Całość zajmuje  $\Theta(n)$ , jeśli zajęcia są już posortowane po czasie zakończenia. Ostatecznie złożoność algorytmu wynosi (uwzględniając początkowe sortowanie):

$$\mathcal{O}(n \log n) + \Theta(n) = \mathcal{O}(n \log n)$$

## Wykład 7

### Kody Huffmana

Często znaki są reprezentowane przy użyciu 8 bitów (np. ASCII). Skoro częstość znaków się różni, lepiej reprezentować znaki występujące często przy użyciu krótszych ciągów bitów, a znaki występujące rzadko przy użyciu dłuższych ciągów bitów.

częstość = liczba wystąpień

**Przykład** W pliku jest 58 znaków. Każdy znak jest elementem zbioru  $\{a, e, i, s, t, r, n\}$ .

|       |                                | <i>a</i> | <i>e</i> | <i>i</i> | <i>s</i> | <i>t</i> | <i>r</i> | <i>n</i> |
|-------|--------------------------------|----------|----------|----------|----------|----------|----------|----------|
|       | częstość                       | 10       | 15       | 12       | 3        | 4        | 13       | 1        |
| kod 1 | stała długość słowa kodowego   | 000      | 001      | 010      | 011      | 100      | 101      | 110      |
| kod 2 | zmienna długość słowa kodowego | 001      | 01       | 10       | 00000    | 0001     | 11       | 00001    |
| kod 3 | zmienna długość słowa kodowego | 001      | 01       | 10       | 00010    | 0001     | 11       | 00001    |

Jeśli użyjemy kodu o stałej długości słowa kodowego (kod 1) to zakodowany plik ma:  $58 \cdot 3 = 174$  bity.

Jeśli użyjemy kodu o zmiennej długości słowa kodowego (kod 2 lub kod 3) to zakodowany plik ma:

$$10 \cdot 3 + 15 \cdot 2 + 12 \cdot 2 + 3 \cdot 5 + 4 \cdot 4 + 13 \cdot 2 + 1 \cdot 5 = 146 \text{ bitów.}$$

Jeśli używamy kodu o zmiennej długości słowa kodowego, to chcemy aby spełniony był warunek:  
**żadne słowo kodowe nie jest prefixem innego słowa kodowego.**

Takie kody nazywa się kodami prefixowymi (choć powinno być bezprefixowe).

Rozważmy kod 3. Ciąg 0001001 można odkodować na dwa sposoby:

- 00010 01  $\Rightarrow s e$
- 0001 001  $\Rightarrow t a$

Tutaj 0001 jest prefixem 00010, czyli słowo kodowe znaku *t* jest prefixem słowa kodowego znaku *s*.

**Reprezentacja drzewowa kodów** Słowo kodowe znaku jest ciągiem etykiet krawędzi na ścieżce od korzenia do wierzchołka zaetykietowanego tym znakiem.

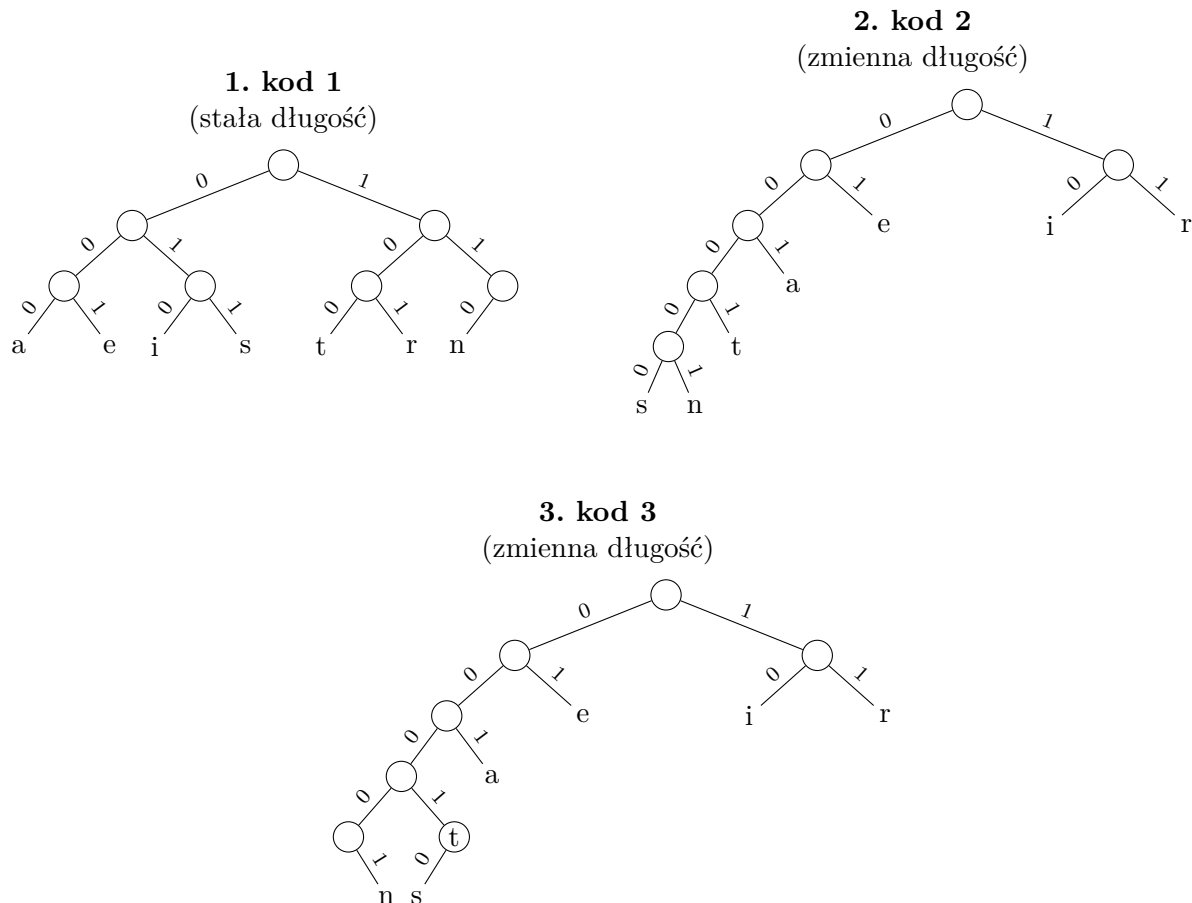
*Uwaga.* Żadne słowo kodowe nie jest prefixem innego słowa kodowego  $\iff$  wszystkie znaki są reprezentowane przez liście w reprezentacji drzewowej kodu.

Etykietujemy wierzchołki w następujący sposób: każdy wierzchołek *v* dostaje etykietę będącą sumą wystąpień znaków, które są reprezentowane przez liście poddrzewa ukorzenionego w *v*.

Niech *C* będzie alfabetem,  $f : C \rightarrow \mathbb{N}_+$ ,  $f(c)$  = częstość znaku  $c \in C$ .

$d_T(c)$  – głębokość liścia reprezentującego znak  $c \in C$  w drzewie *T*

$d_T(c)$  = długość słowa kodowego znaku  $c \in C$  w kodzie reprezentowanym przez drzewo *T*



Liczba bitów w zakodowanym pliku (kod reprezentowany przez drzewo  $T$ ) wynosi:

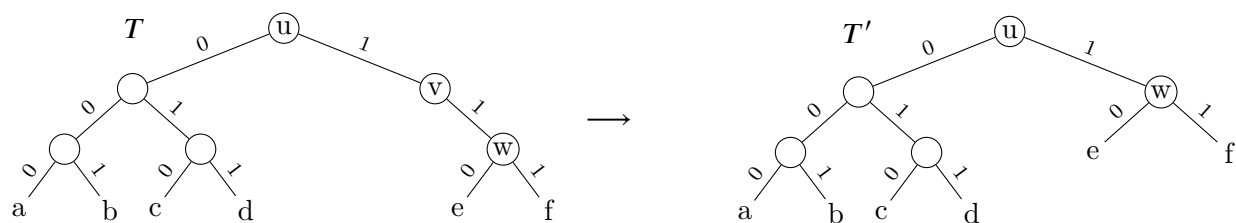
$$B(T) = \sum_{c \in C} f(c) \cdot d_T(c)$$

Kod prefixowy reprezentowany przez drzewo  $T$  nazywamy optymalnym, jeśli  $B(T)$  jest najmniejsze możliwe.

Drzewo ukorzenione nazywamy binarnym, jeśli każdy wierzchołek ma co najwyżej dwoje dzieci. Drzewo binarne nazywamy regularnym, jeśli każdy wierzchołek wewnętrzny ma dokładnie dwoje dzieci.

**Twierdzenie 5.1.** *Drzewo binarne reprezentujące optymalny kod jest regularne.*

*Dowód.* Niech  $T$  będzie drzewem binarnym reprezentującym optymalny kod prefixowy. Przypuśćmy, że  $T$  nie jest regularne. Wtedy istnieje wierzchołek wewnętrzny  $v$  z dokładnie jednym dzieckiem  $w$ . Niech  $u$  będzie rodzicem  $v$  (jeśli  $v$  nie jest korzeniem  $T$ ).



Niech  $T'$  będzie drzewem powstałym z  $T$  przez usunięcie  $v$  i dodanie krawędzi  $uw$ , jeśli  $v$  nie

był korzeniem  $T$ .  $T'$  reprezentuje kod prefixowy i  $B(T') < B(T)$  – sprzeczność z optymalnością  $T$ .  $\square$

## Algorytm Huffmana

Algorytm konstruuje drzewo reprezentujące optymalny kod prefixowy dla  $(C, f)$ . Zaczynamy od  $|C|$  wierzchołków izolowanych (które staną się liśćmi) i wykonujemy  $|C| - 1$  operacji łączenia zaczynając od najrzadszych znaków (będą miały największą głębokość w drzewie).

Będziemy używać kolejki priorytetowej z kluczem  $f(c)$ . Kolejkę priorytetową możemy zaimplementować używając kopca binarnego. Wtedy koszty operacji wynoszą:

- INSERT –  $\mathcal{O}(\log n)$
- EXTRACT\_MIN –  $\mathcal{O}(\log n)$
- CREATE –  $\mathcal{O}(n)$

gdzie  $n$  jest liczbą elementów w kolejce.

### Algorytm 14: Algorytm Huffmana

```

function HUFFMAN( $C, f$ )
1:   $n \leftarrow |C|$ 
2:   $Q \leftarrow \text{CREATE}(C, f)$ 
3:  for  $i \leftarrow 1$  to  $n - 1$  do
4:       $z \leftarrow \text{NEW\_NODE}$ 
5:       $x \leftarrow \text{LEFT}(z) \leftarrow \text{EXTRACT\_MIN}(Q)$ 
6:       $y \leftarrow \text{RIGHT}(z) \leftarrow \text{EXTRACT\_MIN}(Q)$ 
7:       $f(z) \leftarrow f(x) + f(y)$ 
8:      INSERT( $Q, z$ )
9:  return EXTRACT_MIN( $Q$ )
    
```

$\triangleright Q$  - kolejka priorytetowa

Algorytm konstruuje drzewo binarne.  $z$  - wierzchołek, *left, right* - dzieci.

*Dygresja.* Intuicyjnie algorytm działa zachłannie, budując drzewo *od dołu*. Każdy znak  $c \in C$  jest na początku osobnym, izolowanym wierzchołkiem; będzie on liściem gotowego drzewa. Kolejka priorytetowa  $Q$  przechowuje korzenie wszystkich dotąd zbudowanych poddrzew, uporządkowane wg ich łącznej częstości  $f$  (im rzadszy znak, tym wcześniej zostanie wyjęty).

W każdej z  $n - 1$  iteracji pętli wykonujemy jeden krok łączenia:

- EXTRACT\_MIN( $Q$ ) dwukrotnie wybiera dwa korzenie o *najmniejszej* częstości ( $x$  oraz  $y$ );
- tworzymy nowy wierzchołek  $z$  i podpinamy  $x, y$  jako jego lewe i prawe dziecko;
- częstość nowego wierzchołka ustalamy jako  $f(z) = f(x) + f(y)$ , po czym wkładamy  $z$  z powrotem do kolejki przez INSERT( $Q, z$ ).

Łącząc zawsze dwa najrzadsze poddrzewa, spychamy najrzadsze znaki najgłębiej w drzewie – to one dostaną najdłuższe kody, a znaki częste pozostaną blisko korzenia i otrzymają kody krótkie. Każde łączenie zmniejsza liczbę poddrzew w  $Q$  o jeden (dwa wyjęcia, jedno wstawienie), więc po  $n - 1$  iteracjach zostaje dokładnie jeden korzeń – całe drzewo kodu, które zwracamy.

## Złożoność czasowa

- linia 1 –  $\mathcal{O}(1)$
- linia 2 –  $\mathcal{O}(n)$
- wewnątrz pętli z linii 3 wykona się  $n - 1$  razy:
  - linia 4 –  $\mathcal{O}(1)$

- linie 5, 6 –  $\mathcal{O}(\log n)$
- linia 7 –  $\mathcal{O}(1)$
- linia 8 –  $\mathcal{O}(\log n)$

Wnętrze pętli zajmuje łącznie czas  $\mathcal{O}(\log n)$ .

$$\mathcal{O}(n) + (n - 1) \cdot \mathcal{O}(\log n) = \mathcal{O}(n \log n)$$

Złożoność czasowa algorytmu wynosi  $\mathcal{O}(n \log n)$ .

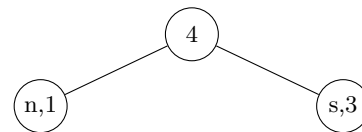
### Przykład 5.2 (Konstrukcja drzewa Huffmana).

Zaczynamy ze zbiorem wierzchołków:  $(n, 1), (s, 3), (t, 4), (a, 10), (i, 12), (r, 13), (e, 15)$ .

1. Łączymy  $n$  i  $s$ .

Zostają:

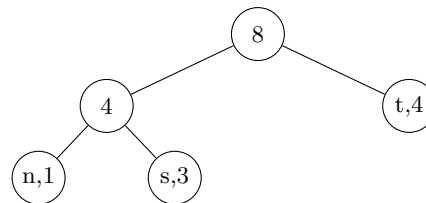
$(t, 4), (a, 10), (i, 12), (r, 13), (e, 15)$ .



2. Łączymy 4 i  $t$ .

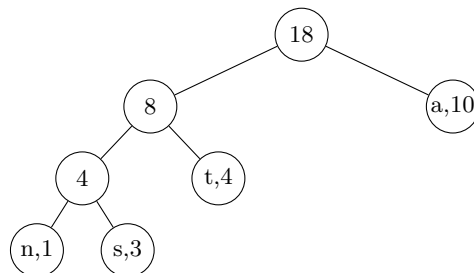
Zostają:

$(a, 10), (i, 12), (r, 13), (e, 15)$ .



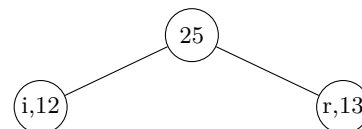
3. Łączymy 8 i  $a$ .

Zostają:  $(i, 12), (r, 13), (e, 15)$ .

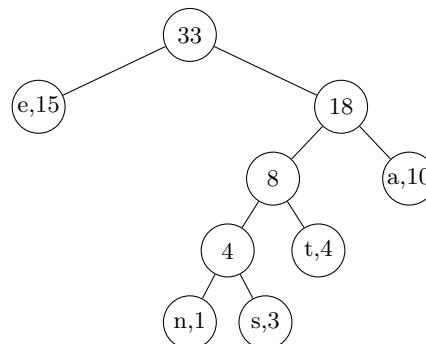


4. Łączymy  $i$  i  $r$ .

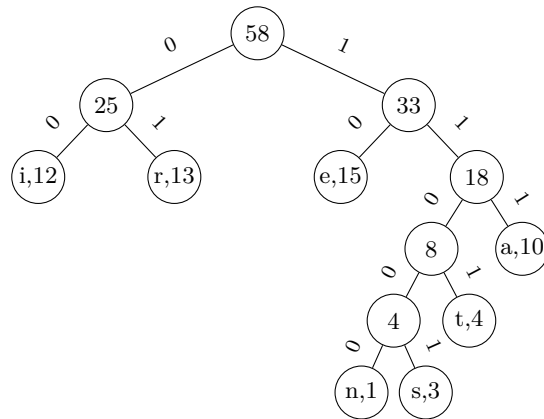
Zostaje:  $(e, 15)$ .



5. Łączymy  $e$  i 18.



6. Łączymy 25 i 33.  
(ostateczne drzewo kodów)



Odczytany kod z drzewa w kroku 6:

$i \Rightarrow 00$   
 $r \Rightarrow 01$   
 $e \Rightarrow 10$   
 $n \Rightarrow 11000$   
 $s \Rightarrow 11001$   
 $t \Rightarrow 1101$   
 $a \Rightarrow 111$

## Wykład 8

### Dowód optymalności algorytmu Huffmana

Kody prefiksowe  $\iff$  regularne drzewa binarne.

$C$  – alfabet,  $f : C \rightarrow \mathbb{N}$  funkcja częstości,  $d_T(c)$  – głębokość liścia reprezentującego  $c$  w drzewie  $T$  reprezentującym kod prefiksowy.

$$B(T) = \sum_{c \in C} f(c) \cdot d_T(c) \longleftarrow \min$$

**Lemat 6.1.** Niech  $x$  i  $y$  będą najrzadziej występującymi znakami z  $C$ . Istnieje optymalny kod prefiksowy, w którym słowa kodowe  $x$  i  $y$  mają tę samą długość i różnią się tylko na ostatnim bicie.

*Dowód.* Znajdziemy drzewo reprezentujące optymalny kod prefiksowy, w którym liście reprezentujące  $x$  i  $y$  mają wspólnego rodzica i mają największą głębokość w drzewie.

Niech  $T$  będzie dowolnym drzewem reprezentującym optymalny kod prefiksowy i niech  $a$  i  $b$  będą dwoma liśćmi o tym samym rodzicu i najmniejszej głębokości w  $T$ . Bez straty ogólności możemy przyjąć, że  $f(x) \leq f(y)$  i  $f(a) \leq f(b)$ .  $f(x)$  i  $f(y)$  są najmniejszymi kluczami, więc  $f(x) \leq f(a)$  i  $f(y) \leq f(b)$ .

Niech  $T'$  będzie drzewem powstałym z  $T$  przez zamianę liści  $x$  i  $a$ .

$$B(T) - B(T') = \sum_{c \in C} f(c) \cdot d_T(c) - \sum_{c \in C} f(c) \cdot d_{T'}(c) =$$

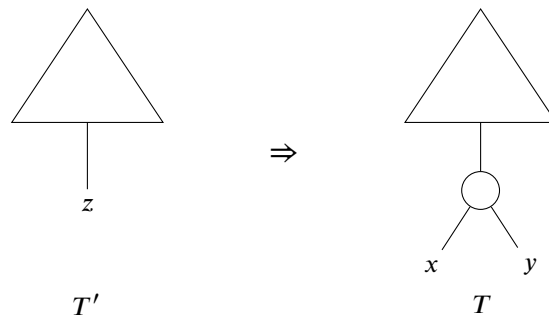
$$\begin{aligned}
 &= f(a) \cdot d_T(a) + f(x) \cdot d_T(x) - f(a) \cdot d_{T'}(a) - f(x) \cdot d_{T'}(x) = \\
 &= f(a) \cdot d_T(a) + f(x) \cdot d_T(x) - f(a) \cdot d_T(x) - f(x) \cdot d_T(a) = \\
 &= f(a)(d_T(a) - d_T(x)) - f(x)(d_T(a) - d_T(x)) = \\
 &= \underbrace{(f(a) - f(x))}_{\geq 0} \underbrace{(d_T(a) - d_T(x))}_{\geq 0} \geq 0
 \end{aligned}$$

Zatem  $B(T) \geq B(T')$ , ale  $B(T)$  najmniejsze możliwe, więc  $B(T') = B(T)$  czyli  $T'$  też reprezentuje optymalny kod prefiksowy. Analogicznie zamieniamy miejscami liście  $y$  i  $b$  i otrzymujemy drzewo  $T''$  reprezentujące optymalny kod prefiksowy, które ma żądane własności.  $\square$

Z lematu 6.1 wynika, że można zacząć konstrukcję optymalnego drzewa połączeniem dwóch symboli o najmniejszej częstości. **Zachłanny wybór** – w każdym kroku algorytm łączy dwa wierzchołki o najmniejszych kluczach.

**Lemat 6.2.** Niech  $A$  będzie alfabetem z funkcją częstości  $f : A \rightarrow \mathbb{N}$ , a  $x, y \in A$  — dwoma znakami o najmniejszych częstościach. Zdefiniujmy zredukowany alfabet  $A' = (A \setminus \{x, y\}) \cup \{z\}$ , gdzie  $z \notin A$ , z funkcją częstości  $f'$  taką, że  $f'(c) = f(c)$  dla  $c \neq z$  oraz  $f'(z) = f(x) + f(y)$ .

Jeśli  $T'$  jest optymalnym drzewem kodu prefiksowego dla  $(A', f')$ , to drzewo  $T$  powstałe z  $T'$  przez zastąpienie liścia  $z$  wewnętrznym węzłem z dziećmi  $x$  i  $y$  jest optymalnym drzewem kodu prefiksowego dla  $(A, f)$ .



*Dowód.* Dla  $c \in A \setminus \{x, y\}$ ,  $d_T(c) = d_{T'}(c)$  a  $d_T(x) = d_T(y) = d_{T'}(z) + 1$ . Zatem:

$$\begin{aligned}
 B(T) &= \sum_{c \in A} f(c) \cdot d_T(c) = \sum_{c \in A \setminus \{x, y\}} f(c) \cdot d_T(c) + f(x) \cdot d_T(x) + f(y) \cdot d_T(y) = \\
 &= \sum_{c \in A \setminus \{x, y\}} f'(c) \cdot d_{T'}(c) + f(x)(d_{T'}(z) + 1) + f(y)(d_{T'}(z) + 1) = \\
 &= \sum_{c \in A \setminus \{x, y\}} f'(c) \cdot d_{T'}(c) + d_{T'}(z)(f(x) + f(y)) + f(x) + f(y) = \\
 &= \sum_{c \in A' \setminus \{z\}} f'(c) \cdot d_{T'}(c) + d_{T'}(z) \cdot f'(z) + f(x) + f(y) = \\
 &= \sum_{c \in A'} f'(c) \cdot d_{T'}(c) + f(x) + f(y) = B(T') + f(x) + f(y)
 \end{aligned}$$

Stąd  $B(T') = B(T) - f(x) - f(y)$ .

Przypuśćmy, że  $T$  nie reprezentuje optymalnego kodu dla  $(A, f)$ . Niech  $T''$  będzie drzewem reprezentującym kod prefiksowy dla  $(A, f)$  takim, że  $B(T'') < B(T)$ . Z lematu 6.1 możemy

wybrać  $T''$  tak, aby  $x$  i  $y$  miały wspólnego rodzica i największą głębokość w  $T''$ . Niech  $T'''$  będzie drzewem powstałym z  $T''$  poprzez usunięcie liści  $x$  i  $y$ , nadanie ich rodzicowi etykiety  $z$  z kluczem  $f(x) + f(y)$ .  $T'''$  reprezentuje kod prefiksowy dla  $(A', f')$ .

Takim samym obliczeniem jak wcześniej (zamieniając wszystkie  $T$  na  $T''$ ,  $T'$  na  $T'''$ ) pokazujemy, że  $B(T''') = B(T'') - f(x) - f(y)$ . Mamy:

$$B(T''') = B(T'') - f(x) - f(y) < B(T) - f(x) - f(y) = B(T')$$

Sprzeczność z optymalnością  $T'$  dla  $(A', f')$ . Zatem  $T$  reprezentuje optymalny kod dla  $(A, f)$ .  $\square$

**Twierdzenie 6.3.** *Algorytm Huffmana konstruuje drzewo reprezentujące optymalny kod prefiksowy.*

*Dowód.*

Indukcja po  $n$  = liczba symboli w alfabecie  $C$ . Dla  $n = 1, 2$  teza jest oczywista  $\checkmark$ . Niech  $n > 2$ .

Po pierwszej operacji połączenia dostajemy alfabet  $C' = (C \setminus \{x, y\}) \cup \{z\}$  i funkcję  $f' : C' \rightarrow \mathbb{N}$  taką, że  $f'(c) = f(c)$  dla wszystkich  $c \in C' \setminus \{z\}$  i  $f'(z) = f(x) + f(y)$ , gdzie  $x$  i  $y$  są znakami z  $C$  o najmniejszych kluczach. Mamy  $|C'| = n - 1$ . Z założenia indukcyjnego algorytm znajduje drzewo  $T'$  reprezentujące optymalny kod prefiksowy dla  $(C', f')$ . Na mocy lematu 6.2, zastąpienie liścia  $z$  w  $T'$  wierzchołkiem z dziećmi  $x$  i  $y$  (co właśnie robi algorytm Huffmana) daje drzewo  $T$  będące optymalnym kodem prefiksowym dla  $(C, f)$ .  $\square$

## Szeregowanie zadań

**Problem 6.4 (Szeregowanie zadań – jeden procesor).**

Jak uszeregować zadania tak, aby średni czas ukończenia był jak najmniejszy?

| $i$ | zadanie | $t_i$ |
|-----|---------|-------|
| 1   | $z_1$   | 15    |
| 2   | $z_2$   | 8     |
| 3   | $z_3$   | 3     |
| 4   | $z_4$   | 10    |

**Przykład 6.5 (Szeregowanie zadań – jeden procesor).**

Rozważmy uszeregowanie  $z_1, z_2, z_3, z_4$ :

$$\frac{15 + (15 + 8) + (15 + 8 + 3) + (15 + 8 + 3 + 10)}{4} = \frac{15 + 23 + 26 + 36}{4} = \frac{100}{4} = 25$$

Rozważmy uszeregowanie  $z_3, z_2, z_4, z_1$ :

$$\frac{3 + (3 + 8) + (3 + 8 + 10) + (3 + 8 + 10 + 15)}{4} = \frac{3 + 11 + 21 + 36}{4} = 17\frac{3}{4}$$

**Rozwiązanie:** Uszereguj zadania od najkrótszego do najdłuższego.

**Problem 6.6 (Szeregowanie zadań – 3 procesory).**

Mamy 3 procesory. Pytanie jak poprzednio (por. 6.5).

| Zadanie | $z_1$ | $z_2$ | $z_3$ | $z_4$ | $z_5$ | $z_6$ | $z_7$ | $z_8$ | $z_9$ |
|---------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| $t_i$   | 3     | 5     | 6     | 10    | 11    | 14    | 15    | 18    | 20    |

**Przykład 6.7 (Szeregowanie zadań – trzy procesory).**

**Rozwiązanie:** Przydzielaj zadania do pierwszego wolnego procesora w kolejności od najkrótszego do najdłuższego.

|       |       |   |       |    |    |       |       |    |    |
|-------|-------|---|-------|----|----|-------|-------|----|----|
|       | 3     | 5 | 6     | 13 | 16 | 20    | 28    | 34 | 40 |
| $P_1$ | $z_1$ |   | $z_4$ |    |    | $z_7$ |       |    |    |
| $P_2$ | $z_2$ |   | $z_5$ |    |    | $z_8$ |       |    |    |
| $P_3$ | $z_3$ |   | $z_6$ |    |    |       | $z_9$ |    |    |

W problemie 6.5 dla uszeregowania  $z_{i_1}, z_{i_2}, \dots, z_{i_n}$  suma zakończeń zadań wynosi:

$$\begin{aligned} \sum_{j=1}^n \sum_{k=1}^j t_{i_k} &= t_{i_1} + (t_{i_1} + t_{i_2}) + (t_{i_1} + t_{i_2} + t_{i_3}) + \dots + (t_{i_1} + \dots + t_{i_n}) = \\ &= n \cdot t_{i_1} + (n-1)t_{i_2} + (n-2)t_{i_3} + \dots + 1 \cdot t_{i_n} = \sum_{j=1}^n (n+1-j)t_{i_j} \end{aligned}$$

W problemie 6.6 (dla  $3 \mid n$ ) dla uszeregowania  $z_{i_1}, z_{i_2}, \dots, z_{i_n}$  suma czasów zakończeń wynosi:

$$\begin{aligned} &\underbrace{t_{i_1} + t_{i_2} + t_{i_3}} + \underbrace{(t_{i_1} + t_{i_4}) + (t_{i_2} + t_{i_5}) + (t_{i_3} + t_{i_6})} + \underbrace{(t_{i_1} + t_{i_4} + t_{i_7}) + (t_{i_2} + t_{i_5} + t_{i_8}) + (t_{i_3} + t_{i_6} + t_{i_9})} + \\ &+ \dots + \underbrace{(t_{i_1} + t_{i_4} + \dots + t_{i_{n-2}}) + (t_{i_2} + t_{i_5} + \dots + t_{i_{n-1}}) + (t_{i_3} + t_{i_6} + \dots + t_{i_n})} = \\ &= \frac{n}{3}(t_{i_1} + t_{i_2} + t_{i_3}) + \left(\frac{n}{3} - 1\right)(t_{i_4} + t_{i_5} + t_{i_6}) + \left(\frac{n}{3} - 2\right)(t_{i_7} + t_{i_8} + t_{i_9}) + \dots + 1 \cdot (t_{i_{n-2}} + t_{i_{n-1}} + t_{i_n}) = \\ &= \sum_{j=1}^n \left(\frac{n}{3} - \left\lfloor \frac{j-1}{3} \right\rfloor\right) t_{i_j} = \sum_{j=1}^n \left\lceil \frac{n+1-j}{3} \right\rceil \cdot t_{i_j} \end{aligned}$$

**Lemat 6.8.** Niech  $A_1 \geq A_2 \geq \dots \geq A_n$ . Dla dowolnej permutacji  $\Pi = (i_1, \dots, i_n)$  zbioru  $[n]$  niech  $f(\Pi) = \sum_{j=1}^n A_j \cdot t_{i_j}$ .  $f(\Pi)$  jest najmniejsze dla dowolnej permutacji  $\Pi = (i_1, \dots, i_n)$  takiej, że  $t_{i_1} \leq t_{i_2} \leq \dots \leq t_{i_n}$ .

*Dowód.* Niech  $\Pi = (i_1, \dots, i_n)$  będzie permutacją minimalizującą  $f$ . Przypuśćmy, że dla pewnych  $k, l \in [n]$ ,  $k < l$  ale  $t_{i_k} > t_{i_l}$ . Niech  $\Pi' = (i'_1, i'_2, \dots, i'_n)$  będzie permutacją powstałą z  $\Pi$  poprzez zamianę  $i_k$  z  $i_l$ , tzn.  $i'_j = i_j$  dla  $j \in [n] \setminus \{k, l\}$ ,  $i'_k = i_l$ ,  $i'_l = i_k$ . Wtedy:

$$f(\Pi) - f(\Pi') = \sum_{j=1}^n A_j \cdot t_{i_j} - \sum_{j=1}^n A_j \cdot t_{i'_j} = A_k \cdot t_{i_k} + A_l \cdot t_{i_l} - A_k \cdot t_{i'_k} - A_l \cdot t_{i'_l} =$$

$$\begin{aligned}
&= A_k \cdot t_{i_k} + A_l \cdot t_{i_l} - A_k \cdot t_{i_l} - A_l \cdot t_{i_k} = A_k(t_{i_k} - t_{i_l}) - A_l(t_{i_k} - t_{i_l}) = \\
&= \underbrace{(t_{i_k} - t_{i_l})}_{>0} \cdot \underbrace{(A_k - A_l)}_{\geq 0} \geq 0
\end{aligned}$$

Zatem  $f(\Pi') \leq f(\Pi)$ , ale  $\Pi$  minimalizuje  $f$ , więc  $f(\Pi') = f(\Pi)$  i  $\Pi'$  też minimalizuje  $f$ . Po pewnej liczbie takich zamian otrzymujemy permutację  $\Pi_0 = (j_1, \dots, j_n)$  minimalizującą  $f$  taką, że  $t_{j_1} \leq t_{j_2} \leq \dots \leq t_{j_n}$ .  $\square$

**Wniosek 6.9.** *Nasze algorytmy szeregowania zadań są poprawne.*

## Wykład 9

### Matroidy

**Definicja 7.1.** Matroid to para  $M = (S, \mathcal{A})$  taka, że:

1.  $S$  – niepusty zbiór skończony,
2.  $\mathcal{A} \subseteq 2^S$  i  $\mathcal{A} \neq \emptyset$  – czyli  $\mathcal{A}$  to niepusta rodzina podzbiorów  $S$ ,
3.  $\forall A, B \subseteq S \quad A \in \mathcal{A} \wedge B \subseteq A \implies B \in \mathcal{A}$ ,
4. własność wymiany:

$$\forall A, B \subseteq S \quad A, B \in \mathcal{A} \wedge |B| > |A| \implies \text{istnieje } x \in B \setminus A \text{ taki, że } A \cup \{x\} \in \mathcal{A}.$$

$\mathcal{A}$  – rodzina zbiorów niezależnych matroidu  $M$ , elementy  $\mathcal{A}$  – zbiory niezależne matroidu  $M$ .

### Przykład 7.2.

1.  $A$  – macierz,  $S$  – zbiór wierszy  $A$ ,  $\mathcal{A} = \{R \subseteq S : R \text{ tworzy zbiór liniowo niezależny}\}$ .  
Wtedy  $(S, \mathcal{A})$  to matroid.
2.  $G = (V, E)$  – graf,  $\mathcal{A} = \{F \subseteq E : G[F] \text{ jest acykliczny}\}$ .  
Wtedy  $(E, \mathcal{A})$  to matroid zwany matroidem grafowym.

Z definicji  $\emptyset \in \mathcal{A}$ .

**Lemat 7.3.** *Niech  $M = (S, \mathcal{A})$ . Wszystkie maksymalne zbiory niezależne  $M$  mają tę samą liczbę.*

*Dowód.* Niech  $A, B \in \mathcal{A}$  będą maksymalnymi zbiorami niezależnymi  $M$ . Przypuśćmy, że  $|B| > |A|$ .

Z własności 4. istnieje  $x \in B \setminus A$  taki, że  $A \cup \{x\} \in \mathcal{A}$  – sprzeczność z maksymalnością  $A$ .  $\square$

**Problem 7.4.**  $M = (S, \mathcal{A})$  – matroid,  $w : S \rightarrow \mathbb{R}_+$  – funkcja wagi.

Dla  $A \subseteq S$  niech  $w(A) = \sum_{x \in A} w(x)$  – waga  $A$ .

Znaleźć zbiór niezależny o maksymalnej wadze w  $M$ .

$S = \{x_1, x_2, \dots, x_n\}$ .

**Algorytm 15: Algorytm Greedy**

```

function GREEDY( $M, w$ )
1:   $n \leftarrow |S|$ 
2:   $A \leftarrow \emptyset$   $\triangleright A$  – rozwiązanie
3:  posortuj  $S$  w porządku nierosnącym względem wagi, tak żeby  $w(x_1) \geq w(x_2) \geq \dots \geq w(x_n)$ 
4:  for  $i \leftarrow 1$  to  $n$  do
5:    if  $A \cup \{x_i\} \in \mathcal{A}$  then
6:       $A \leftarrow A \cup \{x_i\}$ 
7:  return  $A$ 

```

**Twierdzenie 7.5 (Rado, Edmonds).**

Algorytm Greedy znajduje zbiór niezależny o maksymalnej wadze.

*Dowód.* Niech  $A = \{x_{i_1}, x_{i_2}, \dots, x_{i_k}\}$  będzie zbiorem zwróconym przez algorytm i  $w(x_{i_1}) \geq w(x_{i_2}) \geq \dots \geq w(x_{i_k})$ .

Najpierw pokażemy, że  $A$  jest maksymalnym zbiorem niezależnym.

Przypuśćmy, że istnieje  $x_j \in S \setminus A$  taki, że  $A \cup \{x_j\} \in \mathcal{A}$ . Wtedy  $j < i_k$ , bo w przeciwnym wypadku algorytm dodałby  $x_j$  do  $A$  jako  $(k+1)$ -szy element.

Niech  $l$  będzie najmniejszą liczbą taką, że  $j < i_l$ . Wtedy  $w(x_{i_1}) \geq w(x_{i_2}) \geq \dots \geq w(x_{i_{l-1}}) \geq w(x_j) \geq w(x_{i_l})$ .

$A \cup \{x_j\} \in \mathcal{A}$ , więc z własności 3.  $\{x_{i_1}, x_{i_2}, \dots, x_{i_{l-1}}, x_j\} \subseteq A \cup \{x_j\}$  jest zbiorem niezależnym.

Ale wtedy algorytm dodałby  $x_j$  zamiast  $x_{i_l}$  jako  $l$ -ty element dodany do  $A$  – sprzeczność.

Zatem  $A$  jest maksymalnym zbiorem niezależnym.

Rozważmy zbiór niezależny  $B = \{y_1, y_2, \dots, y_m\} \in \mathcal{A}$  i załóżmy, że  $w(y_1) \geq w(y_2) \geq \dots \geq w(y_m)$ .

Mamy  $m \leq k$ , bo  $A$  jest maksymalnym zbiorem niezależnym (Lemat 7.3).

Pokażemy, że  $w(B) \leq w(A)$ .

Pokażemy, że dla każdego  $j \leq m$   $w(y_j) \leq w(x_{i_j})$ .

Przypuśćmy, że  $w(y_j) > w(x_{i_j})$ .

Rozważmy zbiory niezależne  $A_{j-1} = \{x_{i_1}, \dots, x_{i_{j-1}}\} \subseteq A$ ,  $B_j = \{y_1, \dots, y_j\} \subseteq B$ .

$|B_j| = j > |A_{j-1}| = j-1$ , więc z własności 4. istnieje  $y_s \in B_j \setminus A_{j-1}$  taki, że  $A_{j-1} \cup \{y_s\} \in \mathcal{A}$ .

Mamy  $w(y_s) \geq w(y_j) > w(x_{i_j})$ , więc istnieje  $t \leq j$  takie, że  $w(x_{i_1}) \geq w(x_{i_2}) \geq \dots \geq w(x_{i_{t-1}}) \geq w(y_s) > w(x_{i_t})$ .

Z własności 3.  $\{x_{i_1}, x_{i_2}, \dots, x_{i_{t-1}}, y_s\} \subseteq A_{j-1} \cup \{y_s\}$  jest zbiorem niezależnym.

Ale wtedy algorytm dodałby  $y_s$  zamiast  $x_{i_t}$  jako  $t$ -ty element dodany do  $A$  – sprzeczność.

$$\text{Zatem } \left. \begin{array}{l} w(y_1) \leq w(x_{i_1}) \\ w(y_2) \leq w(x_{i_2}) \\ \vdots \\ w(y_m) \leq w(x_{i_m}) \end{array} \right\} \Rightarrow w(B) = w(y_1) + \dots + w(y_m) \leq w(x_{i_1}) + \dots + w(x_{i_m}) \leq w(A). \quad \square$$

**Złożoność czasowa algorytmu Greedy**  $n = |S|$ 

- linia 1:  $\mathcal{O}(1)$
- linia 2:  $\mathcal{O}(1)$
- linia 3:  $\mathcal{O}(n \log n)$
- liczba iteracji pętli z linii 4 wynosi  $n$
- linia 5:  $f(n)$  – czas sprawdzenia warunku z linii 5
- linia 6:  $\mathcal{O}(1)$
- linia 7:  $\mathcal{O}(1)$

**Problem szeregowania zadań**

- $S = \{z_1, z_2, \dots, z_n\}$  – zbiór zadań o jednostkowym czasie wykonania
- $d_1, d_2, \dots, d_n$  – deadlines,  $d_i \in \{1, \dots, n\}$  dla  $i \in [n]$
- $w_1, w_2, \dots, w_n$  – kary,  $w_i \geq 0$  dla  $i \in [n]$

Płacimy karę  $w_i$  za zadanie  $z_i$ , jeżeli nie jest skończone w momencie  $d_i$ .

**Problem 7.6 (szeregowania zadań z karami).**

Znaleźć kolejność wykonania zadań minimalizującą sumę kar.

Założmy, że mamy pewien harmonogram. Zadanie  $z_i$  nazywamy terminowym w tym harmonogramie, jeśli jest zakończone do chwili  $d_i$ , w przeciwnym razie nazywamy je spóźnionym.

Nie płacimy kar za zadania terminowe, płacimy kary za zadania spóźnione.

**Lemat 7.7.** *Każdy harmonogram można przetransformować na harmonogram z taką samą sumą kar, w którym wszystkie zadania terminowe poprzedzają wszystkie zadania spóźnione oraz zadania terminowe są posortowane w kolejności niemalejącej względem deadline'ów.*

*Dowód.*

1. Niech  $z_i$  będzie zadaniem spóźnionym ukończonym w chwili  $l_i > d_i$  i niech  $z_j$  będzie zadaniem terminowym ukończonym w chwili  $l_j \leq d_j$  i niech  $z_i$  będzie przed  $z_j$  w harmonogramie, tzn.  $l_i < l_j$ .

Zamieniamy miejscami  $z_i$  i  $z_j$ . Po zamianie:

- $z_i$  jest ukończone w chwili  $l_j > l_i > d_i$ , więc nadal jest spóźnione.
- $z_j$  jest ukończone w chwili  $l_i < l_j \leq d_j$ , więc nadal jest terminowe.

Po pewnej liczbie takich zamian otrzymujemy harmonogram o tej samej sumie kar, w którym wszystkie zadania terminowe poprzedzają wszystkie zadania spóźnione.

2. Niech  $z_i, z_j$  będą zadaniami terminowymi takimi, że  $z_i$  jest ukończone w chwili  $l_i \leq d_i$ ,  $z_j$  jest ukończone w chwili  $l_j \leq d_j$ ,  $l_i < l_j$ , ale  $d_i > d_j$ .

Zamieniamy miejscami  $z_i$  i  $z_j$ . Po zamianie:

- $z_i$  jest ukończone w chwili  $l_j \leq d_j < d_i$ , więc  $z_i$  jest nadal terminowe.
- $z_j$  jest ukończone w chwili  $l_i < l_j \leq d_j$ , więc  $z_j$  jest nadal terminowe.

Po pewnej liczbie takich zamian otrzymujemy harmonogram spełniający warunki lematu.

□

**Wykład 10****Problem szeregowania zadań (cd.)**

Zbiór  $A \subseteq S$  nazywamy wolnym od kar, jeśli istnieje harmonogram zbioru  $A$ , w którym wszystkie zadania są terminowe.

Dla  $t \in \{0, 1, \dots, n\}$  niech

$$N_t(A) = |\{z_i \in A : d_i \leq t\}|$$

– liczba zadań w  $A$  o deadline'ie  $\leq t$ .

**Lemat 8.1.** Niech  $A \subseteq S$ . Następujące warunki są równoważne:

1.  $A$  jest wolny od kar.
2.  $N_t(A) \leq t$  dla każdego  $t \in \{1, 2, \dots, n\}$ .
3. Jeśli zadania z  $A$  są posortowane w kolejności niemalejącej ze względu na deadliny, to żadne zadanie nie jest spóźnione.

**Twierdzenie 8.2.** Niech  $S$  – zbiór zadań o jednostkowym czasie wykonania, a  $\mathcal{A}$  – rodzina zbiorów wolnych od kar. Wtedy  $(S, \mathcal{A})$  jest matroidem.

*Dowód.* Sprawdzamy własności z definicji matroidu:

1. Oczywiście. ✓
2. Dla każdego  $i \in [n]$  zachodzi  $\{z_i\} \in \mathcal{A}$ , więc  $\mathcal{A} \neq \emptyset$ . ✓
3. Niech  $A \in \mathcal{A}$ ,  $B \subseteq A$ . Istnieje harmonogram zbioru  $A$ , w którym wszystkie zadania są terminowe. Harmonogram zbioru  $B$  powstały z usunięcia z tego harmonogramu zadań z  $A \setminus B$  jest harmonogramem zbioru  $B$ , w którym wszystkie zadania są terminowe. Zatem  $B \in \mathcal{A}$ .
4. Niech  $A, B \in \mathcal{A}$ ,  $|B| > |A|$ .  
Niech  $k \in \{0, 1, \dots, n\}$  będzie największą liczbą  $t$  taką, że  $N_t(B) \leq N_t(A)$ .  
 $k$  jest dobrze zdefiniowane, bo  $N_0(B) = 0 = N_0(A)$ .  
Mamy  $N_n(B) = |B| > |A| = N_n(A)$ , więc  $k < n$ .  
Dla  $j \in \{k+1, k+2, \dots, n\}$  mamy  $N_j(B) > N_j(A)$ , więc  $N_j(B) \geq N_j(A) + 1$ .

$$\left. \begin{array}{l} N_{k+1}(B) > N_{k+1}(A) \\ N_k(B) \leq N_k(A) \end{array} \right\} \implies \text{istnieje zadanie } z \in B \setminus A \text{ o deadline } = k+1$$

Niech  $A' = A \cup \{z\}$ . Pokażemy, że  $A'$  jest wolny od kar, używając lematu 8.1.

- Dla  $t \in \{1, 2, \dots, k\}$ :  $N_t(A') = N_t(A) \leq t$ , bo  $A$  jest wolny od kar.
- Dla  $t \in \{k+1, k+2, \dots, n\}$ :  $N_t(A') = N_t(A) + 1 \leq N_t(B) \leq t$ , bo  $B$  jest wolny od kar.

Zatem  $A'$  jest wolny od kar.

Pokazaliśmy, że  $(S, \mathcal{A})$  jest matroidem. □

Możemy do rozwiązania problemu szeregowania zadań użyć algorytmu Greedy (Algorytm 15) dla matroidu  $(S, \mathcal{A})$  maksymalizującego  $\sum_{z_i \in A} w_i$  (czyli minimalizującego sumę kar zadań spóźnionych).

Z twierdzenia 7.5 (Rado, Edmonds) algorytm Greedy użyty do tego problemu zwraca rozwiązanie optymalne. Warunek z linii 5 algorytmu Greedy (tzn. czy  $A \cup \{x_i\} \in \mathcal{A}$ ) można sprawdzić w czasie  $\mathcal{O}(n)$  – lemat 8.1, warunek 3.

Złożoność czasowa algorytmu wynosi:

$$\mathcal{O}(n \log n + n \cdot n) = \mathcal{O}(n^2)$$

|       |    |    |    |    |    |    |    |    |
|-------|----|----|----|----|----|----|----|----|
| $i$   | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  |
| $d_i$ | 3  | 2  | 2  | 3  | 1  | 4  | 6  | 5  |
| $w_i$ | 80 | 70 | 60 | 50 | 40 | 30 | 20 | 10 |

**Przykład 8.3 (Szeregowanie zadań z deadlineami).**

Zadania są już posortowane w porządku nierosnącym względem wagi ( $w_1 \geq w_2 \geq \dots \geq w_8$ ). Kolejne iteracje algorytmu Greedy (poniżej  $A$  wypisane w kolejności niemalejącej po deadlineach):

1.  $A = (z_1)$
2.  $A = (z_2, z_1)$
3.  $A = (z_2, z_3, z_1)$
4.  $A = (z_2, z_3, z_1)$  ( $z_4$  nie dodajemy, bo  $N_3(A \cup \{z_4\}) = 4 > 3$ )
5.  $A = (z_2, z_3, z_1)$  ( $z_5$  nie dodajemy, bo  $N_2(A \cup \{z_5\}) = 3 > 2$ )
6.  $A = (z_2, z_3, z_1, z_6)$
7.  $A = (z_2, z_3, z_1, z_6, z_7)$
8.  $A = (z_2, z_3, z_1, z_6, z_8, z_7)$

Harmonogram (zadania terminowe posortowane po deadlineach, potem zadania spóźnione):

$(z_2, z_3, z_1, z_6, z_8, z_7, z_4, z_5)$

Suma kar  $= w_4 + w_5 = 50 + 40 = 90$

**Algorytmy aproksymacyjne**

Algorytmy aproksymacyjne stosuje się do problemów, dla których jest mała szansa na znalezienie algorytmu dokładnego (np. wielomianowego). Zamiast szukać rozwiązania optymalnego, szukamy rozwiązania „prawie optymalnego”.

Niech  $\Pi$  – problem optymalizacyjny znalezienia maksimum lub minimum pewnej funkcji celu  $f$ .

- $D_\Pi$  – zbiór instancji problemu  $\Pi$ .
- $I \in D_\Pi$  – instancja  $\Pi$ .
- $S(I)$  – zbiór rozwiązań dopuszczalnych dla instancji  $I$ .
- $f : \bigcup_{I \in D_\Pi} S(I) \rightarrow \mathbb{R}$  – funkcja celu. Możemy założyć, że

$$\forall I \in D_\Pi \quad \forall s \in S(I) \quad f(s) > 0$$

Niech  $c^*(I)$  oznacza wartość rozwiązania optymalnego instancji  $I$ ,  $c^*(I) > 0$ .

$A$  – algorytm aproksymacyjny rozwiązujący  $\Pi$ .

$c(I)$  – wartość rozwiązania znalezionej przez algorytm  $A$  dla instancji  $I$ ,  $c(I) > 0$ .

**Definicja 8.4.** Mówimy, że algorytm  $A$  ma współczynnik aproksymacji  $\rho(n)$ , jeśli dla każdej instancji  $I \in D_\Pi$  rozmiaru  $n$  zachodzi

$$\max \left\{ \frac{c(I)}{c^*(I)}, \frac{c^*(I)}{c(I)} \right\} \leq \rho(n)$$

Wtedy  $A$  nazywamy algorytmem  $\rho(n)$ -aproksymacyjnym.

Zauważmy, że:

$$\begin{aligned} \text{dla problemu maksymalizacji: } & 0 < c(I) \leq c^*(I) \\ \text{dla problemu minimalizacji: } & 0 < c^*(I) \leq c(I) \end{aligned} \quad \Rightarrow \quad \max \left\{ \frac{c(I)}{c^*(I)}, \frac{c^*(I)}{c(I)} \right\} \geq 1$$

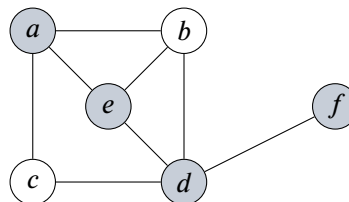
Niech  $\rho = \sup_{n \in \mathbb{N}} \{\rho(n)\}$ . Wtedy mówimy, że współczynnik aproksymacji algorytmu  $A$  wynosi  $\rho$  i  $A$  nazywamy algorytmem  $\rho$ -aproksymacyjnym.

**Problem pokrycia wierzchołkowego**

**Definicja 8.5.** Niech  $G = (V, E)$  – graf. Pokryciem wierzchołkowym (Vertex Cover, VC) grafu  $G$  nazywamy zbiór  $W \subseteq V$  taki, że dla każdej krawędzi  $e \in E$  zachodzi  $W \cap e \neq \emptyset$  – czyli każda krawędź  $G$  ma co najmniej jeden koniec w  $W$ .

**Problem 8.6 (pokrycia wierzchołkowego).** • Instancja: graf  $G$ .

- Problem: znaleźć pokrycie wierzchołkowe grafu  $G$  o minimalnej liczbie wierzchołków.



**Przykład 8.7 (Pokrycie wierzchołkowe).**

$W = \{a, e, d, f\}$  jest pokryciem wierzchołkowym powyższego grafu  $G$  (zaznaczone wyróżnione wierzchołki).

### Algorytm Approx Vertex Cover

#### Algorytm 16: Approx Vertex Cover

```

function AVC( $G$ )
1:   $C \leftarrow \emptyset$ 
2:   $F \leftarrow E(G)$ 
3:  while  $F \neq \emptyset$  do
4:    wybierz dowolną krawędź  $uv \in F$ 
5:     $C \leftarrow C \cup \{u, v\}$ 
6:    usuń z  $F$  wszystkie krawędzie incydentne z  $u$  lub z  $v$ 
7:  return  $C$ 
    
```

▷  $C$  – rozwiązanie  
▷  $F$  – zbiór dotychczas niepokrytych krawędzi

**Przykład 8.8 (Algorytm AVC – przykład).**

Rozważmy graf z poprzedniego przykładu.

Pierwszy przebieg algorytmu może wybrać kolejno krawędzie  $ab$  i  $cd$ :

- Po wybraniu  $ab$  usuwamy z  $F$  krawędzie  $ab, ac, ae, bd, be$ , zostają  $cd, de, df$ .
- Po wybraniu  $cd$  usuwamy z  $F$  krawędzie  $cd, de, df$  –  $F = \emptyset$ .

Algorytm zwraca  $C = \{a, b, c, d\}$ .

Inny przebieg algorytmu może wybrać kolejno krawędzie  $df$ ,  $be$  i  $ac$ :

- Po wybraniu  $df$  usuwamy z  $F$  krawędzie  $df, de, bd, cd$ .
- Po wybraniu  $be$  usuwamy z  $F$  krawędzie  $be, ae, ab$ .
- Po wybraniu  $ac$  usuwamy ostatnią krawędź  $ac$  –  $F = \emptyset$ .

Algorytm zwraca  $C = \{d, f, e, b, a, c\}$ .

## Wykład 11

### Algorytm Approx Vertex Cover (cd.)

**Złożoność czasowa algorytmu AVC**  $n = |V(G)|$ ,  $m = |E(G)|$ .

- linia 1:  $\mathcal{O}(1)$
- linia 2:  $\mathcal{O}(m)$
- pętlę z linii 3–6 można zaimplementować, żeby działała w czasie  $\mathcal{O}(m)$
- linia 7:  $\mathcal{O}(1)$

Złożoność czasowa AVC wynosi  $\mathcal{O}(m)$ .

**Twierdzenie 9.1.** *AVC jest algorytmem 2-aproksymacyjnym.*

*Dowód.* Niech  $G$  – graf wejściowy.

$c^*(G)$  – najmniejsza liczba wierzchołków w pokryciu wierzchołkowym  $G$ .

$c(G)$  – liczba wierzchołków w pokryciu wierzchołkowym znalezionym przez AVC.

Niech  $B$  będzie zbiorem krawędzi wybranych przez AVC w linii 4.

$B$  jest skojarzeniem (czyli zbiorem parami rozłącznych krawędzi), bo po wybraniu krawędzi  $uv$  algorytm usuwa z  $F$  wszystkie krawędzie incydentne z  $u$  lub  $v$ .

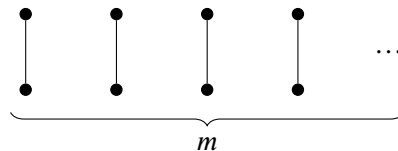
Żeby pokryć krawędzie z  $B$  potrzeba co najmniej  $|B|$  wierzchołków. Stąd  $c^*(G) \geq |B|$ .

$c(G) = 2|B|$ , bo algorytm dodaje do  $C$  oba końce każdej krawędzi z  $B$ .

$$c(G) = 2|B| \leq 2 \cdot c^*(G) \implies \frac{c(G)}{c^*(G)} \leq 2$$

□

Pokażemy, że ograniczenie 2 jest osiągnięte. Rozważmy graf złożony z  $m$  rozłącznych krawędzi:

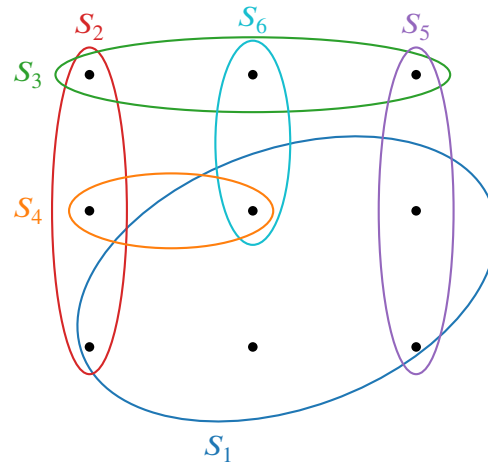


AVC zwraca pokrycie złożone z  $2m$  wierzchołków, natomiast optymalne pokrycie ma rozmiar  $m$  (po jednym końcu z każdej krawędzi). Stąd  $\frac{c(G)}{c^*(G)} = 2$ .

### Problem pokrycia zbioru

**Definicja 9.2.** Niech  $X$  – zbiór skończony,  $|X| = n$ ,  $\mathcal{F} \subseteq 2^X$  – rodzina podzbiorów zbioru  $X$  taka, że  $\bigcup_{S \in \mathcal{F}} S = X$  (każdy element  $X$  należy do co najmniej jednego zbioru z  $\mathcal{F}$ ).

Rodzina  $\mathcal{A} \subseteq \mathcal{F}$  jest pokryciem zbioru  $X$ , jeśli  $\bigcup_{S \in \mathcal{A}} S = X$ .



### Przykład 9.3 (Pokrycie zbioru).

$\mathcal{F} = \{S_1, S_2, S_3, S_4, S_5, S_6\}$ .  $\{S_1, S_2, S_3\}$  – pokrycie.

**Problem 9.4 (pokrycia zbioru).** • Instancja:  $(X, \mathcal{F})$ , gdzie  $X$  – zbiór skończony,  $|X| = n$ ,  $\mathcal{F} \subseteq 2^X$  taki, że  $\bigcup_{S \in \mathcal{F}} S = X$ .  
• Problem: znaleźć pokrycie  $\mathcal{A} \subseteq \mathcal{F}$  minimalizujące  $|\mathcal{A}|$ .

*Uwaga.* VC jest szczególnym przypadkiem problemu pokrycia zbiorów. Dla  $G = (V, E)$  kładziemy  $X = E$ ,  $\mathcal{F} = \{E(v) : v \in V\}$ , gdzie  $E(v) = \{e \in E : v \in e\}$ .

### Algorytm Greedy Set Cover

#### Algorytm 17: Greedy Set Cover

```

function GSC( $X, \mathcal{F}$ )
1:   $U \leftarrow X$                                  $\triangleright U$  – zbiór jeszcze niepokrytych elementów
2:   $\mathcal{A} \leftarrow \emptyset$                          $\triangleright \mathcal{A}$  – rozwiązanie
3:  while  $U \neq \emptyset$  do
4:    wybierz  $S \in \mathcal{F}$  maksymalizujący  $|S \cap U|$ 
5:     $\mathcal{A} \leftarrow \mathcal{A} \cup \{S\}$ 
6:     $U \leftarrow U \setminus S$ 
7:  return  $\mathcal{A}$ 
    
```

**Poprawność** Pętla obraca się dopóki są jeszcze niepokryte elementy, a w każdym obrocie pętli jakiś jeszcze niepokryte elementy zostają pokryte.

### Złożoność czasowa

- linia 1:  $\mathcal{O}(n)$
- linia 2:  $\mathcal{O}(1)$
- liczba iteracji pętli 3–6 jest  $\leq |X|$  i  $\leq |\mathcal{F}|$ , więc jest  $\leq \min\{|X|, |\mathcal{F}|\}$ . Wnętrze pętli:
  - linia 4:  $\mathcal{O}(|X| \cdot |\mathcal{F}|)$
  - linia 5:  $\mathcal{O}(1)$
  - linia 6:  $\mathcal{O}(|X|)$
 łącznie  $\mathcal{O}(|X| \cdot |\mathcal{F}|)$ .
- linia 7:  $\mathcal{O}(1)$

Ostatecznie złożoność GSC wynosi  $\mathcal{O}(|X| \cdot |\mathcal{F}| \cdot \min\{|X|, |\mathcal{F}|\})$ .

**Twierdzenie 9.5.** *GSC jest algorytmem  $H(\max\{|S| : S \in \mathcal{F}\})$ -aproxymacyjnym, gdzie*

$$H(0) = 0, \quad H(m) = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{m} \text{ dla } m \geq 1$$

*Dowód.* Niech  $(X, \mathcal{F})$  będzie instancją wejściową.

$\mathcal{A}^*$  – optymalne pokrycie dla  $(X, \mathcal{F})$ .

$\mathcal{A}$  – pokrycie dla  $(X, \mathcal{F})$  znalezione przez GSC.

Niech  $S_i$  będzie zbiorem wybranym przez algorytm w linii 4 w  $i$ -tej iteracji pętli. Przyznajemy koszt

$$c_x = \frac{1}{|S_i \setminus (S_1 \cup S_2 \cup \dots \cup S_{i-1})|}$$

każdemu elementowi  $x \in S_i \setminus (S_1 \cup \dots \cup S_{i-1})$  ( $S_i \setminus (S_1 \cup \dots \cup S_{i-1})$  to zbiór elementów pokrytych po raz pierwszy w  $i$ -tej iteracji pętli). Całkowity koszt przyznany podczas jednej iteracji pętli wynosi 1. Skoro  $|\mathcal{A}|$  jest liczbą iteracji pętli, to

$$|\mathcal{A}| = \sum_{x \in X} c_x$$

Rozważmy sumę  $\sum_{S \in \mathcal{A}^*} \sum_{x \in S} c_x$ . Skoro  $\mathcal{A}^*$  jest pokryciem, każdy  $x \in X$  występuje w tej sumie co najmniej raz, więc

$$\sum_{S \in \mathcal{A}^*} \sum_{x \in S} c_x \geq \sum_{x \in X} c_x = |\mathcal{A}|$$

Pokażemy, że  $\sum_{x \in S} c_x \leq H(|S|)$  dla każdego  $S \in \mathcal{F}$ .

Niech  $S \in \mathcal{F}$ . Dla  $i \in \{1, 2, \dots, |\mathcal{A}|\}$  niech  $n_i = |S \setminus (S_1 \cup \dots \cup S_i)|$  (liczba elementów w  $S$ , które nie są pokryte po  $i$ -tej iteracji pętli) i niech  $n_0 = |S|$ .

Niech  $k$  będzie najmniejszą liczbą taką, że  $n_k = 0$ . Wtedy  $S$  jest pokryty przez  $S_1 \cup S_2 \cup \dots \cup S_k$ . Oczywiście  $n_{i-1} \geq n_i$  dla  $i \in [k]$ , a  $n_{i-1} - n_i$  to liczba elementów z  $S$  pokrytych po raz pierwszy przez  $S_i$ .

Zatem

$$\sum_{x \in S} c_x = \sum_{i=1}^k (n_{i-1} - n_i) \cdot \frac{1}{|S_i \setminus (S_1 \cup \dots \cup S_{i-1})|}$$

Na podstawie wyboru zachłannego w linii 4 mamy  $|S_i \setminus (S_1 \cup \dots \cup S_{i-1})| \geq |S \setminus (S_1 \cup \dots \cup S_{i-1})| = n_{i-1}$  dla każdego  $S \in \mathcal{F} \setminus \{S_1, \dots, S_{i-1}\}$ .

Mamy:

$$\begin{aligned} \sum_{x \in S} c_x &= \sum_{i=1}^k (n_{i-1} - n_i) \cdot \frac{1}{|S_i \setminus (S_1 \cup \dots \cup S_{i-1})|} \leq \sum_{i=1}^k (n_{i-1} - n_i) \cdot \frac{1}{n_{i-1}} \\ &= \sum_{i=1}^k \sum_{j=n_i+1}^{n_{i-1}} \frac{1}{n_{i-1}} \leq \sum_{i=1}^k \sum_{j=n_i+1}^{n_{i-1}} \frac{1}{j} = \sum_{i=1}^k \left( \sum_{j=1}^{n_{i-1}} \frac{1}{j} - \sum_{j=1}^{n_i} \frac{1}{j} \right) \\ &= \sum_{i=1}^k (H(n_{i-1}) - H(n_i)) \\ &= (H(n_0) - H(n_1)) + (H(n_1) - H(n_2)) + \dots + (H(n_{k-1}) - H(n_k)) \end{aligned}$$

$$= H(n_0) - H(n_k) = H(|S|) - H(0) = H(|S|).$$

Ostatecznie mamy:

$$\begin{aligned} |\mathcal{A}| &= \sum_{x \in X} c_x \leq \sum_{S \in \mathcal{A}^*} \sum_{x \in S} c_x \leq \sum_{S \in \mathcal{A}^*} H(|S|) \leq \sum_{S \in \mathcal{A}^*} \max\{H(|S|) : S \in \mathcal{A}^*\} \\ &= |\mathcal{A}^*| \cdot \max\{H(|S|) : S \in \mathcal{A}^*\} \\ &\leq |\mathcal{A}^*| \cdot \max\{H(|S|) : S \in \mathcal{F}\} = |\mathcal{A}^*| \cdot H(\max\{|S| : S \in \mathcal{F}\}), \end{aligned}$$

więc  $\frac{|\mathcal{A}|}{|\mathcal{A}^*|} \leq H(\max\{|S| : S \in \mathcal{F}\})$ . □

**Wniosek 9.6.** *GSC jest algorytmem  $(\ln |X| + 1)$ -aproksymacyjnym.*

*Dowód.*  $H(n) \leq \ln n + 1$ , więc

$$H(\max\{|S| : S \in \mathcal{F}\}) \leq H(|X|) \leq \ln |X| + 1$$

□

VC jest szczególnym przypadkiem problemu pokrycia zbiorów. Dla  $G = (V, E)$  kładziemy  $X = E$ ,  $\mathcal{F} = \{E(v) : v \in V\}$ , gdzie  $E(v) = \{e \in E : v \in e\}$ ,  $|E(v)| = d(v)$ . Wtedy  $\max\{|E(v)| : E(v) \in \mathcal{F}\} = \Delta(G)$ , więc GSC zastosowany do VC ma współczynnik aproksymacji  $H(\Delta(G))$ .

Dla grafów o maksymalnym stopniu  $\leq 3$  ( $H(3) < 2 < H(4)$ ) oszacowanie współczynnika aproksymacyjnego GSC jest lepsze niż oszacowanie dla AVC.

### Schematy aproksymacyjne

Niech  $\Pi$  – problem (NP-trudny).

**Definicja 9.7.** Algorytm  $A$  jest schematem aproksymacyjnym dla  $\Pi$ , jeśli dla dowolnego wejścia  $(I, \epsilon)$ , gdzie  $I$  jest instancją  $\Pi$  i  $\epsilon > 0$ ,  $A$  jest algorytmem  $(1 + \epsilon)$ -aproksymacyjnym.

**Definicja 9.8.** Schemat aproksymacyjny  $A$  nazywamy wielomianowym schematem aproksymacyjnym, jeśli jego złożoność czasowa może być ograniczona wielomianową funkcją względem  $|I|$  dla każdego ustalonego  $\epsilon > 0$ .

Złożoność czasowa wielomianowego schematu aproksymacyjnego może wynosić np.  $\Theta(2^{1/\epsilon} \cdot n^2)$ , gdzie  $n = |I|$ . Wtedy zmniejszanie precyzji (czyli zmniejszanie  $\epsilon$ ) jest kosztowne czasowo.

**Definicja 9.9.** Schemat aproksymacyjny  $A$  nazywamy w pełni wielomianowym schematem aproksymacyjnym, jeśli jego złożoność czasowa może być ograniczona funkcją wielomianową względem  $|I|$  i  $\frac{1}{\epsilon}$ .

## Wykład 12

### Problem sumy podzbiorów

**Problem 10.1 (sumy podzbiorów (Subset-Sum)).**

$S = \{x_1, x_2, \dots, x_n\} \subseteq \mathbb{N}$ ,  $t \in \mathbb{N}$ .

**Wersja decyzyjna**

- Instancja:  $(S, t)$ .
- Problem: czy istnieje  $S' \subseteq S$  takie, że  $\sum_{x_i \in S'} x_i = t$ ?

**Wersja optymalizacyjna**

- Instancja:  $(S, t)$ .
- Problem: znaleźć  $S' \subseteq S$  takie, że  $\sum_{x_i \in S'} x_i$  jest największą możliwą liczbą  $\leq t$ .

Dla zbioru  $P \subseteq \mathbb{N}$  i  $x \in \mathbb{N}$  oznaczamy  $P + x = \{s + x : s \in P\}$ .

np. dla  $P = \{1, 2, 3, 6\}$  i  $x = 2$ :  $P + x = \{3, 4, 5, 8\}$ .

Dla listy  $L$  liczb naturalnych i  $x \in \mathbb{N}$  przez  $L + x$  oznaczamy listę powstałą z  $L$  przez dodanie  $x$  do każdego elementu.

np.  $L = (3, 5, 8)$  i  $x = 3$ :  $L + x = (6, 8, 11)$ .

**Lemat 10.2.** Niech  $S = \{x_1, \dots, x_n\} \subseteq \mathbb{N}$ . Niech  $P_0 = \{0\}$  i  $P_i = P_{i-1} \cup (P_{i-1} + x_i)$  dla  $i \in [n]$ . Zbiór  $P_n$  składa się ze wszystkich możliwych sum podzbiorów zbioru  $S$ .

*Dowód.* Pokażemy przez indukcję po  $i$ , że zbiór  $P_i$  składa się ze wszystkich możliwych sum elementów ze zbioru  $\{x_1, x_2, \dots, x_i\}$ .

$i = 0$ :  $P_0 = \{0\}$  i  $\sum_{x_i \in \emptyset} x_i = 0$ . ✓

Założmy, że  $P_{i-1}$  składa się ze wszystkich możliwych sum elementów z  $\{x_1, \dots, x_{i-1}\}$ .

Niech  $S' \subseteq \{x_1, \dots, x_i\}$ .

- Jeśli  $x_i \notin S'$ , to  $S' \subseteq \{x_1, \dots, x_{i-1}\}$ , więc  $\sum_{x_k \in S'} x_k \in P_{i-1} \subseteq P_i$ .
- Jeśli  $x_i \in S'$ , to  $S' \setminus \{x_i\} \subseteq \{x_1, \dots, x_{i-1}\}$ , więc

$$\sum_{x_k \in S'} x_k = \sum_{x_k \in S' \setminus \{x_i\}} x_k + x_i \in P_{i-1} + x_i \subseteq P_i$$

Zatem  $P_i$  zawiera wszystkie możliwe sumy elementów z  $\{x_1, \dots, x_i\}$ .

$P_i$  nie zawiera innych elementów, bo  $P_{i-1}$  składa się z sum elementów z  $\{x_1, \dots, x_{i-1}\}$  a  $P_{i-1} + x_i$  zawiera tylko sumy elementów z  $\{x_1, \dots, x_{i-1}, x_i\} \subseteq \{x_1, \dots, x_i\}$ .

Dla  $i = n$  mamy tezę. □

Założmy, że mamy procedurę MERGELists( $L, L'$ ), która dla posortowanych list liczb naturalnych  $L, L'$  zwraca posortowaną listę elementów z  $L$  i  $L'$  i usuwa duplikaty. Złożoność czasowa MERGELists( $L, L'$ ) wynosi  $\Theta(|L| + |L'|)$ .

**Algorytm 18: Exact Subset Sum**

**function** EXACTSUBSETSUM( $S, t$ )

- 1:  $n \leftarrow |S|$
- 2:  $L_0 \leftarrow (0)$
- 3: **for**  $i \leftarrow 1$  **to**  $n$  **do**
- 4:      $L_i \leftarrow \text{MERGELists}(L_{i-1}, L_{i-1} + x_i)$
- 5:     usuń z  $L_i$  wszystkie liczby  $> t$
- 6: **return** największy element  $L_n$

Poprawność wyniku z lematu 10.2.

**Złożoność czasowa** Długość listy  $L_i$  może być wykładnicza względem  $i$ , nawet  $2^i$ . Dla  $I = (S, t)$ :

$$|I| = \Theta\left(\log t + \sum_{i=1}^n \log x_i\right)$$

Jeśli liczby  $t$  i  $x_i$  są  $n$ -cyfrowe, to  $\log t, \log x_i = \Theta(n)$  i wtedy  $|I| = \Theta(n + n^2) = \Theta(n^2)$ . Skoro  $L_n$  może mieć długość  $2^n$ , to ExactSubsetSum jest algorytmem wykładniczym.

Jeśli liczba  $t$  jest ograniczona przez funkcję wielomianową od  $n = |S|$ , czyli  $t = \mathcal{O}(n^p)$  dla jakiegoś  $p \in \mathbb{N}$ , to długość  $L_i$  jest ograniczona przez  $\mathcal{O}(t) = \mathcal{O}(n^p)$  i w tym przypadku algorytm działa w czasie wielomianowym.

**Schemat aproksymacyjny** Pokażemy w pełni wielomianowy schemat aproksymacyjny dla tego problemu.

**Pomysł:** będziemy skracać listy  $L_i$ . Dla  $0 < \delta < 1$  z listy  $L$  robimy krótszą listę  $L'$  taką, że dla każdego elementu  $y$  usuniętego z  $L$  istnieje element  $z$  w  $L'$ , który jest blisko niego, dokładnie  $\frac{y}{1+\delta} \leq z \leq y$ .

**Przykład 10.3 (Procedura Trim).**

$L = (15, 16, 17, 25, 27, 28, 35, 36, 37)$ ,  $\delta = 0,1$ .

$L' = (15, 17, 25, 28, 35)$ .

Następująca procedura  $\text{TRIM}(L, \delta)$  realizuje ten pomysł.

$L = (y_1, \dots, y_m)$  – posortowana rosnąco lista,  $0 < \delta < 1$ .

#### Algorytm 19: Trim

```

function TRIM( $L, \delta$ )
1:   $m \leftarrow |L|$ 
2:   $L' \leftarrow (y_1)$ 
3:   $last \leftarrow y_1$ 
4:  for  $i \leftarrow 2$  to  $m$  do
5:    if  $y_i > last \cdot (1 + \delta)$  then
6:      dołącz  $y_i$  na koniec  $L'$ 
7:       $last \leftarrow y_i$ 
8:  return  $L'$ 

```

Czas działania  $\text{TRIM}(L, \delta)$  wynosi  $\Theta(|L|)$ .

Oto schemat aproksymacyjny:

#### Algorytm 20: Approx Subset Sum

```

function APPROXSUBSETSUM( $S, t, \epsilon$ )
1:   $n \leftarrow |S|$ 
2:   $L_0 \leftarrow (0)$ 
3:  for  $i \leftarrow 1$  to  $n$  do
4:     $L_i \leftarrow \text{MERGE}(\text{LISTS}(L_{i-1}, L_{i-1} + x_i))$ 

```

|    |                                                         |                                                |
|----|---------------------------------------------------------|------------------------------------------------|
| 5: | usuń z $L_i$ liczby $> t$                               |                                                |
| 6: | $L_i \leftarrow \text{TRIM}(L_i, \frac{\epsilon}{2^n})$ | $\triangleright \delta = \frac{\epsilon}{2^n}$ |
| 7: | <b>return</b> największy element $z \in L_n$            | $\triangleright z^*$                           |

### NP-zupełność problemu sumy podzbiorów

**Twierdzenie 10.4.** *Problem sumy podzbiorów jest NP-zupełny.*

*Dowód.* Problem jest w NP. Dla instancji  $(S, t)$  i  $S' \subseteq S$  można sprawdzić w czasie wielomianowym czy  $\sum_{x_i \in S'} x_i = t$ .

Pokażemy redukcję  $3\text{-SAT} \leq_p \text{SUBSET-SUM}$ .

$\Phi$  – formuła 3-SAT w CNF,  $x_1, x_2, \dots, x_n$  – zmienne,  $C_1, C_2, \dots, C_m$  – klauzule. Załóżmy, że żadna klauzula nie zawiera zmiennej i jej zaprzeczenia (wpp. jest spełniona) oraz każda zmienna występuje w co najmniej jednej klauzuli.

Utworzymy instancję  $(S, t)$  problemu SUBSET-SUM dla  $\Phi$ .

Dla każdej zmiennej  $x_i$  mamy w  $S$  dwie liczby  $r_i$  i  $r'_i$ ,  $(n+m)$ -cyfrowe.

Dla każdej klauzuli  $C_j$  mamy w  $S$  dwie liczby  $s_j$  i  $s'_j$ ,  $(n+m)$ -cyfrowe.

$t$  jest liczbą  $(n+m)$ -cyfrową:  $\underbrace{11 \dots 1}_n \underbrace{44 \dots 4}_m$ .

Pierwsze  $n$  cyfr etykietowane jest zmiennymi, ostatnie  $m$  cyfr etykietowane jest klauzulami.

- $r_i, r'_i$  mają 1 na pozycji  $x_i$  i 0 na innych pozycjach etykietowanych zmiennymi.
- 1 na pozycji  $C_j$ : jeśli  $x_i$  występuje w  $C_j$  to w  $r_i$ , jeśli  $\bar{x}_i$  to w  $r'_i$ .  
Na pozostałych pozycjach etykietowanych klauzulami  $r_i$  i  $r'_i$  mają 0.
- $s_j$  ma na pozycji  $C_j$  1, a na reszcie pozycji 0.
- $s'_j$  ma na pozycji  $C_j$  2, a na reszcie pozycji 0.

Liczby  $r_i, r'_i, s_j, s'_j$  są różne – z konstrukcji i założeń o  $\Phi$ .

Na każdej pozycji suma wszystkich liczb  $\leq 6$ , więc nie ma przeniesień.

Redukcja jest w czasie wielomianowym:  $|S|$  jest  $2(n+m)$  liczb  $(n+m)$ -cyfrowych, utworzenie  $t$  zajmuje czas liniowy względem  $n+m$ , utworzenie każdej z nich jest w czasie wielomianowym względem  $n+m$ .

$\Phi$  spełnialna  $\implies$  istnieje spełniające wartościowanie.

Jeśli  $x_i = 1$ , to do  $S'$  bierzemy  $r_i$ .

Jeśli  $x_i = 0$ , to do  $S'$  bierzemy  $r'_i$ .

$C_j$  spełniona  $\implies$  suma  $r_i, r'_i \in S'$  ma na pozycji  $C_j$  1 lub 2 lub 3.

Dokładamy do  $S'$  odpowiednio  $s_j$  i  $s'_j$  lub  $s'_j$  lub  $s_j$ .

Wtedy  $S'$  ma sumę równą  $t$ .

$S' \subseteq S$  ma sumę równą  $t \implies$  do  $S'$  należy dokładnie jedna z liczb  $r_i, r'_i$ :

- jeśli  $r_i \in S'$ , to  $x_i = 1$ ,
- jeśli  $r'_i \in S'$ , to  $x_i = 0$ .

$C_j$  jest spełniona, bo  $s_j + s'_j \leq 3$  na pozycji  $C_j \implies$  co najmniej 1 jest z  $r_i, r'_i$  należących do  $S'$  mająca 1 na pozycji  $C_j$ :

- $r_i \in S'$  to  $x_i$  jest w  $C_j$ , a mamy  $x_i = 1$ ,
- $r'_i \in S'$  to  $\bar{x}_i$  jest w  $C_j$ , a mamy  $x_i = 0$ .

□

**Przykład 10.5 (Redukcja 3-SAT do SUBSET-SUM).**

$$C_1 = (x_1 \vee \overline{x_2} \vee \overline{x_3}), \quad C_2 = (\overline{x_1} \vee \overline{x_2} \vee \overline{x_3}),$$

$$C_3 = (\overline{x_1} \vee \overline{x_2} \vee x_3), \quad C_4 = (x_1 \vee x_2 \vee x_3).$$

$$\Phi = C_1 \wedge C_2 \wedge C_3 \wedge C_4. \quad n = 3, m = 4.$$

|        | $x_1$ | $x_2$ | $x_3$ | $C_1$ | $C_2$ | $C_3$ | $C_4$ |
|--------|-------|-------|-------|-------|-------|-------|-------|
| $t$    | 1     | 1     | 1     | 4     | 4     | 4     | 4     |
| $r_1$  | 1     | 0     | 0     | 1     | 0     | 0     | 1     |
| $r'_1$ | 1     | 0     | 0     | 0     | 1     | 1     | 0     |
| $r_2$  | 0     | 1     | 0     | 0     | 0     | 0     | 1     |
| $r'_2$ | 0     | 1     | 0     | 1     | 1     | 1     | 0     |
| $r_3$  | 0     | 0     | 1     | 0     | 0     | 1     | 1     |
| $r'_3$ | 0     | 0     | 1     | 1     | 1     | 0     | 0     |
| $s_1$  | 0     | 0     | 0     | 1     | 0     | 0     | 0     |
| $s'_1$ | 0     | 0     | 0     | 2     | 0     | 0     | 0     |
| $s_2$  | 0     | 0     | 0     | 0     | 1     | 0     | 0     |
| $s'_2$ | 0     | 0     | 0     | 0     | 2     | 0     | 0     |
| $s_3$  | 0     | 0     | 0     | 0     | 0     | 1     | 0     |
| $s'_3$ | 0     | 0     | 0     | 0     | 0     | 2     | 0     |
| $s_4$  | 0     | 0     | 0     | 0     | 0     | 0     | 1     |
| $s'_4$ | 0     | 0     | 0     | 0     | 0     | 0     | 2     |

Dla wartościowania  $x_1 = 1, x_2 = 0, x_3 = 0$  otrzymujemy:

$$S' = \{r_1, r'_2, r'_3, s_1, s'_2, s_3, s_4, s'_4\}$$

$$\text{i } \sum_{x \in S'} x = t.$$

**Wykład 13****Problem sumy podzbioru**

Dane:  $S = \{x_1, x_2, \dots, x_n\} \subseteq \mathbb{N}, t \in \mathbb{N}$ .

Znaleźć:  $S' \subseteq S$  takie, że  $\sum_{x_i \in S'} x_i$  jest największa możliwa, ale  $\leq t$ .

**Algorytm 21: Aproksymacja sumy podzbioru**

**function** APPROXSUBSETSUM( $S, t, \epsilon$ )

1:  $n \leftarrow |S|$

2:  $L_0 \leftarrow (0)$

3:  $\delta \leftarrow \frac{\epsilon}{2n}$

4: **for**  $i \leftarrow 1$  **to**  $n$  **do**

5:      $L_i \leftarrow \text{MERGE}(L_{i-1}, L_{i-1} + x_i)$

6:     usuń z  $L_i$  wszystkie elementy  $> t$

7:      $L_i \leftarrow \text{TRIM}(L_i, \delta)$

8: **return** największy element z  $L_n$

$$\triangleright \delta = \frac{\epsilon}{2n}$$

$\triangleright z^*$  – liczba zwrócona przez algorytm

**Algorytm 22:** Procedura Trim

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                        |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|
| <pre> <b>function</b> TRIM(<math>L, \delta</math>) 1:  <math>m \leftarrow  L </math> 2:  <math>L' \leftarrow (y_1)</math> 3:  <math>last \leftarrow y_1</math> 4:  <b>for</b> <math>i \leftarrow 2</math> <b>to</b> <math>m</math> <b>do</b> 5:    <b>if</b> <math>y_i &gt; last \cdot (1 + \delta)</math> <b>then</b> 6:      dodaj <math>y_i</math> na koniec <math>L'</math> 7:      <math>last \leftarrow y_i</math> 8:  <b>return</b> <math>L'</math> </pre> | $\triangleright L = (y_1, \dots, y_m)$ |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|

**Przykład 11.1 (Aproksymacja sumy podzbioru).**

$S = \{205, 203, 400, 200\}$ ,  $t = 650$ ,  $\epsilon = 0.4$ .  $n = 4$ ,  $\delta = \frac{\epsilon}{2n} = \frac{0.4}{8} = 0.05$ .

$$L_0 = (0)$$

$$L_1 = (0, 205)$$

$$L_2 = (0, 203, 205, 408) \quad (\text{po linii 4})$$

$$L_2 = (0, 203, 205, 408) \quad (\text{po linii 5})$$

$$L_2 = (0, 203, 408) \quad (\text{po linii 6})$$

$$L_3 = (0, 203, 400, 408, 603, 808)$$

$$L_3 = (0, 203, 400, 408, 603)$$

$$L_3 = (0, 203, 400, 603)$$

$$L_4 = (0, 200, 203, 400, 403, 600, 603, 803)$$

$$L_4 = (0, 200, 203, 400, 403, 600, 603)$$

$$L_4 = (0, 200, 400, 600)$$

$$z^* = 600 \quad (600 = 400 + 200)$$

Optymalne rozwiązanie:  $205 + 203 + 200 = 608$ .

$$\frac{608}{600} \leq 1.4 = 1 + \epsilon$$

**Twierdzenie 11.2.** Algorytm APPROXSUBSETSUM jest w pełni wielomianowym schematem aproksymacyjnym dla problemu sumy podzbioru.

*Dowód.* Użyjemy faktów z analizy:

**Fakt 1**  $\left(1 + \frac{\epsilon}{2n}\right)^n$  jest ciągiem rosnącym.

**Fakt 2**  $e^x \leq 1 + x + x^2$  dla  $0 \leq x \leq 1$ .

**Fakt 3**  $\frac{x}{1+x} \leq \ln(1+x)$  dla  $x \geq 0$ .

Oczywiście  $z^* \in P_n$ , więc  $z^*$  jest sumą elementów z  $S$ .

Niech  $y^* \in P_n$  będzie optymalną sumą elementów  $\leq t$ .

Oczywiście  $y^* \geq z^*$ . Pokażemy, że  $\frac{y^*}{z^*} \leq 1 + \epsilon$ .

Pokażemy przez indukcję po  $i$ , że dla każdego  $y \in P_i$ ,  $y \leq t$  istnieje  $z \in L_i$  taki, że:

$$\frac{y}{\left(1 + \frac{\epsilon}{2n}\right)^i} \leq z \leq y \quad (\text{po } i\text{-tym obrocie pętli}) \quad (2)$$

Dla  $i = 1$ :  $P_1 = \{0, x_1\}$ ,  $L_1 = (0, x_1)$  – zachodzi oczywiście.

Założmy, że (2) zachodzi dla jakiegoś  $i < n$ . Pokażemy, że (2) zachodzi dla  $i + 1$ .

Niech  $y \in P_{i+1} = P_i \cup (P_i + x_{i+1})$ .

**Przypadek 1:**  $y \in P_i$ .

Z założenia indukcyjnego istnieje  $z \in L_i$  takie, że  $\frac{y}{\left(1 + \frac{\epsilon}{2n}\right)^i} \leq z \leq y$ .

Jeśli  $z \in L_{i+1}$ , to  $\frac{y}{\left(1 + \frac{\epsilon}{2n}\right)^{i+1}} \leq \frac{y}{\left(1 + \frac{\epsilon}{2n}\right)^i} \leq z \leq y$ , więc (2) zachodzi.

Jeśli  $z \notin L_{i+1}$  (został usunięty przez Trim), to istnieje  $z' \in L_{i+1}$  takie, że  $z' \leq z \leq z' \left(1 + \frac{\epsilon}{2n}\right)$ , więc  $\frac{z}{1 + \frac{\epsilon}{2n}} \leq z' \leq z$ .

Mamy:

$$\frac{y}{\left(1 + \frac{\epsilon}{2n}\right)^{i+1}} = \frac{1}{1 + \frac{\epsilon}{2n}} \frac{y}{\left(1 + \frac{\epsilon}{2n}\right)^i} \stackrel{\text{zał. ind.}}{\leq} \frac{z}{1 + \frac{\epsilon}{2n}} \leq z' \leq z \leq y,$$

więc (2) zachodzi.

**Przypadek 2:**  $y \in P_i + x_{i+1}$ .

Wtedy  $y = y' + x_{i+1}$ , gdzie  $y' \in P_i$ .

Z założenia indukcyjnego istnieje  $z \in L_i$  takie, że  $\frac{y'}{\left(1 + \frac{\epsilon}{2n}\right)^i} \leq z \leq y'$ .

Jeśli  $z + x_{i+1} \in L_{i+1}$ , to

$$\frac{y}{\left(1 + \frac{\epsilon}{2n}\right)^{i+1}} \leq \frac{y}{\left(1 + \frac{\epsilon}{2n}\right)^i} = \frac{y' + x_{i+1}}{\left(1 + \frac{\epsilon}{2n}\right)^i} \leq \frac{y'}{\left(1 + \frac{\epsilon}{2n}\right)^i} + x_{i+1} \leq z + x_{i+1} \leq y' + x_{i+1} = y,$$

więc (2) zachodzi.

Jeśli  $z + x_{i+1} \notin L_{i+1}$  (został usunięty przez Trim), to istnieje  $z' \in L_{i+1}$  takie, że  $z' \leq z + x_{i+1} \leq z' \left(1 + \frac{\epsilon}{2n}\right)$ , więc  $\frac{z + x_{i+1}}{1 + \frac{\epsilon}{2n}} \leq z' \leq z + x_{i+1}$ .

Mamy:

$$\begin{aligned} \frac{y}{\left(1 + \frac{\epsilon}{2n}\right)^{i+1}} &= \frac{1}{1 + \frac{\epsilon}{2n}} \frac{y}{\left(1 + \frac{\epsilon}{2n}\right)^i} = \frac{1}{1 + \frac{\epsilon}{2n}} \frac{y' + x_{i+1}}{\left(1 + \frac{\epsilon}{2n}\right)^i} \\ &\leq \frac{1}{1 + \frac{\epsilon}{2n}} \left( \frac{y'}{\left(1 + \frac{\epsilon}{2n}\right)^i} + x_{i+1} \right) \stackrel{\text{zał. ind.}}{\leq} \frac{1}{1 + \frac{\epsilon}{2n}} (z + x_{i+1}) \\ &\leq z' \leq z + x_{i+1} \leq y' + x_{i+1} = y \end{aligned}$$

Więc (2) zachodzi.

W szczególności (2) zachodzi dla  $y^* \in P_n$ . Zatem istnieje  $z \in L_n$  takie, że  $\frac{y^*}{\left(1 + \frac{\epsilon}{2n}\right)^n} \leq z \leq y^*$ , więc

$$\frac{y^*}{z} \leq \left(1 + \frac{\epsilon}{2n}\right)^n.$$

Ale  $z^* \geq z$ , więc  $\frac{y^*}{z^*} \leq \frac{y^*}{z} \leq \left(1 + \frac{\epsilon}{2n}\right)^n$ .

$$\frac{y^*}{z^*} \leq \left(1 + \frac{\epsilon}{2n}\right)^n \stackrel{\text{Fakt 1}}{\leq} \lim_{n \rightarrow \infty} \left(1 + \frac{\epsilon}{2n}\right)^n = \lim_{n \rightarrow \infty} \left( \left(1 + \frac{\epsilon}{2n}\right)^{\frac{2n}{\epsilon}} \right)^{\frac{\epsilon}{2}} = e^{\frac{\epsilon}{2}}$$

□

Pokażemy, że algorytm działa w czasie wielomianowym względem  $|I| = \Theta(\log t + \sum_{i=1}^n \log x_i)$  i  $\frac{1}{\epsilon}$ .

Na każdej liście  $L_i$  po linii 6 elementy  $z_0 = 0, z_1, \dots, z_k$  spełniają warunek  $z_{j+1} > z_j \left(1 + \frac{\epsilon}{2n}\right)$  dla  $j \geq 1$ . Ile różnych liczb naturalnych  $z_1, \dots, z_k$  spełniających ten warunek może się tu zmieścić w przedziale  $[1, t]$ ?

Przez indukcję po  $i$  mamy, że  $z_{i+1} > z_1 \left(1 + \frac{\epsilon}{2n}\right)^i$  dla  $1 \leq i \leq k-1$ . Zatem

$$z_1 \left(1 + \frac{\epsilon}{2n}\right)^{k-1} < z_k \leq t$$

Skoro  $z_1 \geq 1$ , to

$$\left(1 + \frac{\epsilon}{2n}\right)^{k-1} \leq t \quad \Big| \ln$$

$$(k-1) \ln \left(1 + \frac{\epsilon}{2n}\right) \leq \ln t$$

$$k-1 \leq \frac{\ln t}{\ln \left(1 + \frac{\epsilon}{2n}\right)} \quad \Big| +2$$

$$k+1 \leq \frac{\ln t}{\ln \left(1 + \frac{\epsilon}{2n}\right)} + 2 \stackrel{\text{Fakt 3}}{\leq} \frac{\ln t \cdot \left(1 + \frac{\epsilon}{2n}\right)}{\frac{\epsilon}{2n}} + 2 = \frac{2n + \epsilon}{2n} \cdot \frac{2n}{\epsilon} \ln t + 2 \leq \frac{3n \ln t}{\epsilon} + 2$$

Skoro długość  $L_i$  po linii 6 wynosi  $k+1 \leq \frac{3n \ln t}{\epsilon} + 2$ , a po  $(i+1)$ -szej linii 4 może być co najwyżej 2 razy dłuższa i  $|I| = \Omega(\ln t + n)$ , to złożoność czasowa algorytmu jest wielomianowa względem  $|I|$  i  $\frac{1}{\epsilon}$ .

### Najliczniejsze skojarzenie w grafach

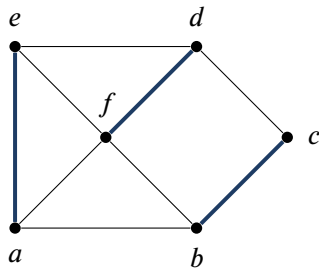
**Najliczniejsze skojarzenie w grafach dwudzielnych**  $G = (V, E)$  – graf.

$G$  jest dwudzielny jeśli istnieje podział  $V$  na dwa niepuste podzbiory  $V_1, V_2$  takie, że dla każdej krawędzi  $e = v_1 v_2 \in E$  mamy  $v_1 \in V_1$  i  $v_2 \in V_2$ . Oznaczamy to jako  $G = (V_1, V_2, E)$ .

Skojarzenie w  $G$  to zbiór parami rozłącznych krawędzi.

Skojarzenie jest najliczniejsze jeśli ma największą liczbę.

#### Przykład 11.3.



Przykładowe skojarzenie:  $\{ae, df, bc\}$

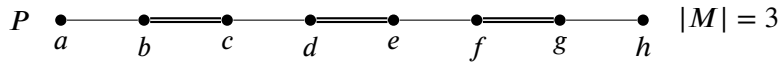
Jeśli  $M$  jest skojarzeniem, to każdy wierzchołek należy do co najwyżej jednej krawędzi z  $M$ .

Niech  $M$  będzie ustalonym skojarzeniem w  $G$ .

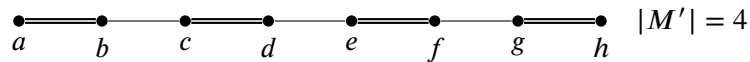
- wierzchołek jest skojarzony jeśli należy do jakiejś krawędzi z  $M$  (mówimy też, że  $M$  pokrywa ten wierzchołek),

- wierzchołek jest wolny jeśli nie jest skojarzony,
- krawędź jest skojarzona jeśli należy do  $M$ ,
- krawędź jest wolna jeśli nie należy do  $M$ .

Ścieżka powiększająca (względem  $M$ ) to naprzemienna ścieżka składająca się z wolnych i skojarzonych krawędzi o wolnych końcach.



$$M' = M \div E(P) = (M - E(P)) \cup (E(P) - M)$$



Możemy użyć ścieżki powiększającej i różnicy symetrycznej do zwiększenia liczności skojarzenia.

#### Twierdzenie 11.4 (Berge'a).

Niech  $G = (V, E)$  – graf,  $M$  – skojarzenie w  $G$ .

$M$  jest najliczniejszym skojarzeniem w  $G \iff$  nie istnieje ścieżka powiększająca względem  $M$ .

*Dowód.*  $\implies$  Przypuśćmy, że istnieje ścieżka powiększająca  $P$  względem  $M$ .

Wtedy  $M' = M \div E(P)$  jest skojarzeniem i  $|M'| > |M|$ , więc  $M$  nie jest skojarzeniem najliczniejszym.

$\impliedby$  Przypuśćmy, że  $M$  nie jest najliczniejszym skojarzeniem w  $G$ .

Niech  $M'$  będzie skojarzeniem takim, że  $|M'| > |M|$ .

Niech  $H$  będzie podgrafem  $G$  indukowanym przez  $M' \div M$ , czyli  $H = (V(G), M' \div M)$ .

$M, M'$  to skojarzenia, więc stopień wierzchołka  $d_H(v) \leq 2$  dla każdego  $v \in V(G)$ .

Każda składowa  $H$  jest ścieżką lub cyklem.

$M, M'$  to skojarzenia, więc nie ma cykli nieparzystych w  $H$ .

Każda składowa  $H$  jest ścieżką lub cyklem parzystym.

Skoro  $|M'| > |M|$ , to istnieje składowa  $H$ , która jest ścieżką zawierającą więcej krawędzi z  $M'$  niż z  $M$ , więc ta ścieżka zaczyna się i kończy krawędzią z  $M'$ . Ta ścieżka jest ścieżką powiększającą względem  $M$ .  $\square$

## Wykład 14

Kontynuujemy temat najliczniejszego skojarzenia w grafach dwudzielnych.

Dane:  $G = (V_1, V_2; E)$  – graf dwudzielny.

Znaleźć: najliczniejsze skojarzenie w  $G$ .

Niech  $M$  będzie skojarzeniem w  $G$ .

Definiujemy graf skierowany  $G_M = (V_1 \cup V_2, E_M)$ , gdzie

$$E_M = \{(u, v) : uv \in E - M, u \in V_1, v \in V_2\} \cup \{(v, u) : uv \in M, u \in V_1, v \in V_2\}.$$

Innymi słowy, krawędzie wolne kierujemy z  $V_1$  do  $V_2$ , a krawędzie skojarzone z  $V_2$  do  $V_1$ .

**Przykład 12.1.**

Procedura do znajdowania ścieżki powiększającej — *Augmenting Path Search* (APS).

**Algorytm 23: Augmenting Path Search**

```

function APS( $G, M$ )
1:   $V'_1 \leftarrow$  zbiór wierzchołków wolnych z  $V_1$ 
2:   $V'_2 \leftarrow$  zbiór wierzchołków wolnych z  $V_2$ 
3:  skonstruuj graf skierowany  $G_M = (V_1 \cup V_2, E_M)$ 
4:  znajdź ścieżkę  $\vec{P}$  z  $V'_1$  do  $V'_2$  w  $G_M$ 
5:  if  $\vec{P}$  istnieje then
6:    return  $P$ 
7:  else
8:    return NULL

```

$\triangleright G = (V_1, V_2; E), M$  – skojarzenie w  $G$

$\triangleright P$  – nieskierowana  $\vec{P}$

**Lemat 12.2.** Procedura APS zwraca ścieżkę  $P \iff$  istnieje ścieżka powiększająca względem  $M$  w  $G$ . Ponadto ścieżka  $P$  zwrócona przez APS jest ścieżką powiększającą względem  $M$ .

*Dowód.*

$\implies$  Jeśli APS zwraca ścieżkę  $P$ , to skierowana ścieżka  $\vec{P}$  znaleziona w linii 4 zaczyna się w wierzchołku z  $V'_1$  (a więc wolnym) i kończy się w wierzchołku z  $V'_2$  (a więc wolnym). Z definicji grafu  $G_M$  wynika, że pierwsza krawędź  $P$  jest wolna, druga krawędź  $P$  jest skojarzona, trzecia krawędź  $P$  jest wolna, ..., ostatnia krawędź  $P$  jest wolna. Zatem  $P$  jest ścieżką powiększającą względem  $M$ .

$\impliedby$  Jeśli istnieje ścieżka powiększająca względem  $M$  w  $G$ , to ścieżka skierowana otrzymana z takiej ścieżki powiększającej przez skierowanie krawędzi zgodnie z  $E_M$  jest ścieżką skierowaną z  $V'_1$  do  $V'_2$  w  $G_M$ . Taką ścieżkę skierowaną znajduje APS.

□

Algorytm znajdowania najliczniejszego skojarzenia — *Maximum Matching* (MM).

**Algorytm 24: Maximum Matching**

```

function MM( $G$ )
1:   $M \leftarrow \emptyset$ 
2:  repeat
3:     $P \leftarrow$  APS( $G, M$ )
4:    if  $P \neq \text{NULL}$  then
5:       $M \leftarrow M \div E(P)$ 
6:  until  $P = \text{NULL}$ 
7:  return  $M$ 

```

$\triangleright G = (V_1, V_2; E)$

**Poprawność** Wynika z twierdzenia Berge'a.

**Złożoność czasowa** Złożoność czasowa APS:

- Linia 1, 2:  $\mathcal{O}(|V|)$ .
- Linia 3:  $\mathcal{O}(|E|)$ .
- Linia 4:  $\mathcal{O}(|E|)$  (używamy algorytmu BFS).
- Linia 5–8:  $\mathcal{O}(|V|)$ .

Złożoność APS wynosi zatem  $\mathcal{O}(|E|)$ .

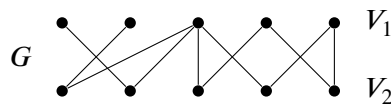
Złożoność czasowa MM:

- Linia 1:  $\mathcal{O}(1)$ .
- Pętla 2–6 wykona się co najwyżej  $\frac{|V|}{2}$  razy, bo w każdej iteracji  $|M|$  zwiększa się o 1, a maksymalna liczność skojarzenia jest  $\leq \frac{|V|}{2}$ .
- Wnętrze pętli:
  - Linia 3:  $\mathcal{O}(|E|)$ .
  - Linia 5:  $\mathcal{O}(|V|)$ .
 Razem  $\mathcal{O}(|E|)$ .
- Linia 7:  $\mathcal{O}(|V|)$ .

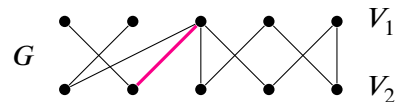
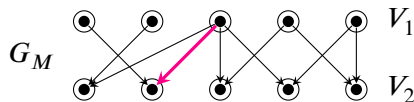
Ostateczna złożoność MM wynosi  $\mathcal{O}(|V| \cdot |E|)$ .

### Przykład 12.3 (Działanie algorytmu Maximum Matching).

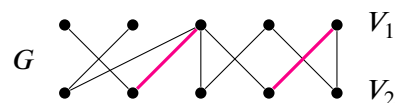
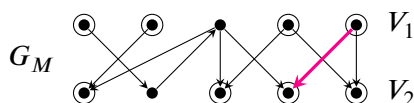
Prześledźmy działanie algorytmu MM na poniższym grafie  $G$ :



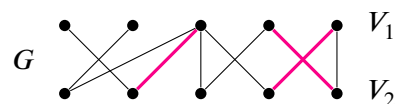
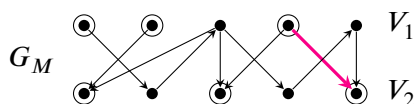
**Iteracja 1:**



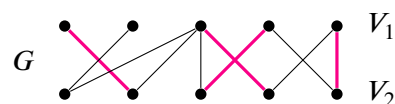
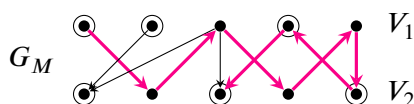
**Iteracja 2:**



**Iteracja 3:**



**Iteracja 4:**



Iteracja 5:

